Softwareentwicklung mit Java 2



2. Vererbung und Dynamische Bindung

Vorbemerkung: Statisch vs. dynamisch

Statisch

- bedeutet wie im Programmtext

Statischer Typ

- Typ der Deklaration

Statische Typprüfung

- durch Compiler (**javac**) auf Basis des Programmtextes
- führen zu Syntaxfehler

Statische Bindung

- Methodenaufrufe werden statisch auf Basis des statischen Typs durch den Compiler aufgelöst

Java ist eine **statisch typisierte** Sprache

Dynamisch

- bedeutet wie bei der Ausführung Programmtext

Dynamischer Typ

- Typ des Werts

Dynamische Typprüfung

- zur Laufzeit auf Basis von konkreten Werten (java)
- führen zu Laufzeitfehler (Exceptions)

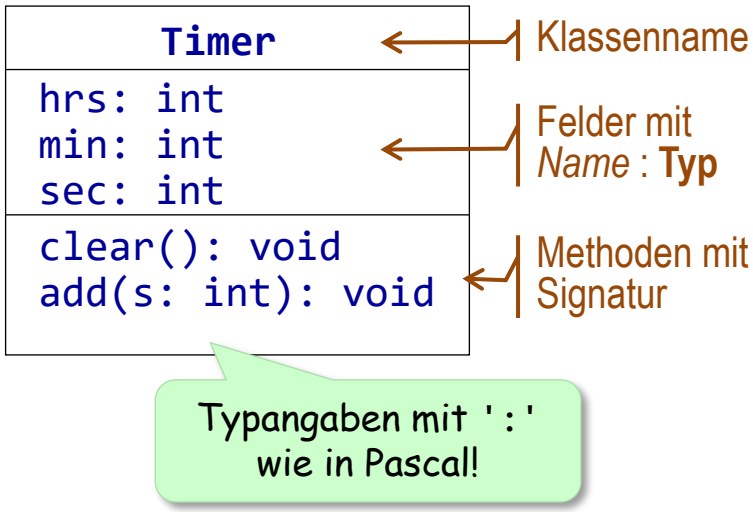
Dynamische Bindung

- Methodenaufrufe werden auf Basis des dynamischen Typs zur Laufzeit aufgelöst

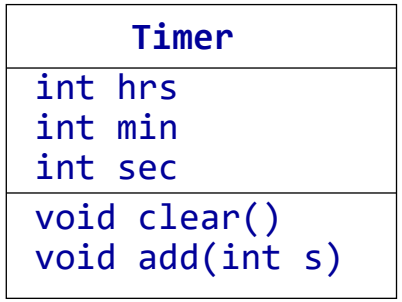
Unified Modeling Language (UML)

- Standard für graphische Darstellung von objektorientierten Programmen
- Mehrere Diagrammarten
  - Klassendiagramm
  - Objektdiagramm
  - Sequenzdiagramm
  - Kollaborationsdiagramm
  - Zustandsdiagramm
  - …

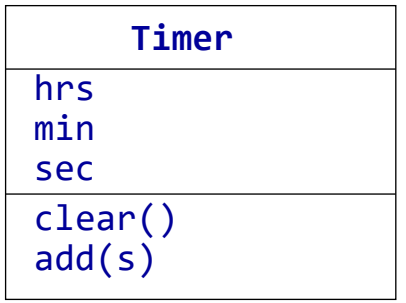
Klassen



Alternativ: Typangaben wie in Java



Verkürzt: ohne Typangaben



Manchmal auch: nur Klassennamen



© H. Prähofer, 2026

2. Vererbung und Dynamische Bindung

2.1 Vererbung bei Klassen

2.2 Typhierarchien und dynamische Bindung

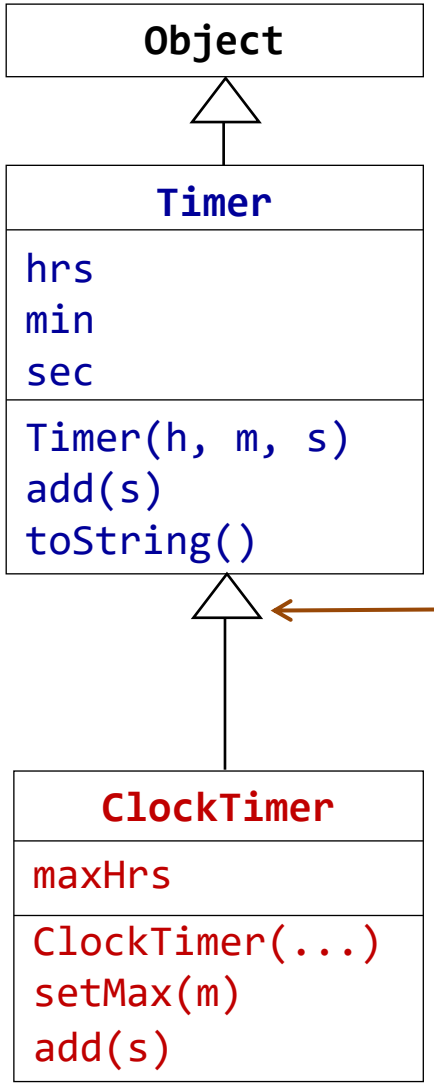
2.3 Kovarianz bei Rückgabewerten und Exceptions

2.4 Fallstudie List

2.5 Entwurfsmuster

© H. Prähofer, 2026

Vererbung (Ableitung, Spezialisierung)



Wurzel *der Vererbungshierarchie*

Oberklasse *(Basisklasse, Superklasse)*

ClockTimer **leitet von** Timer **ab**
ClockTimer **ist Unterklasse** von Timer
ClockTimer **erbt von** Timer
ClockTimer **erweitert** Timer

Unterklasse *(Subklasse, Spezialisierung, Ableitung)*

Felder: `hrs`, `min`, `sec`, `maxHrs`
Methoden: `add(s)`, `toString()`, `setMax(m)`
Konstruktur: `ClockTimer(…)`

Unterklasse erbt
- Felder und
- Methoden der Oberklasse

Unterklasse kann
- zusätzliche Methoden und Felder definieren
- Methoden überschreiben

© H. Prähofer, 2026

Vererbung in Java

- Ableitung von Basisklasse mit **extends**
- Überschreiben von Methoden mit gleicher Schnittstelle
  - Annnotation **@Override** zeigt Überschreiben an
- Überschriebene Methode **kann** mit **Super-Aufruf** aufgerufen werden
- Konstruktor der Subklasse **muss Konstruktor der Superklasse** aufrufen

extends Object

```
public class Timer  {
  protected int hrs, min, sec;

  public Timer(int hrs, int min, int sec) {
    this.hrs = hrs;
    this.min = min;
    this.sec = sec;
  }

  public void add(int s) {
    ...
  }

  @Override
  public String toString() {
    return hrs + ":" + min + ":" + sec;
  }
}
```

```
public class ClockTimer extends Timer {
  private int maxHrs;

  public ClockTimer(int h, int m, int s, int max) {
    super(h, m, s);
    maxHrs = max;
  }

  @Override
  public void add(int s) {
    super.add(s);
    hrs = hrs % maxHrs;
  }

  public void setMax(int max) {
    maxHrs = max;
    hrs = hrs % maxHrs;
  }
}
```

Ableiten von Basisklasse

zusätzliche Felder

Konstruktor mit Aufruf von Konstruktor der Basisklasse

Überschreiben von Methoden mit Annotation **@Override**

Methode der Super-Klasse kann aufgerufen werden

zusätzliche Methoden

© H. Prähofer, 2026

Die Klasse Object

Klasse Object ist Wurzel der Vererbungshierarchie

- alle Klassen erben direkt oder indirekt von **Object**
- erben damit die Methoden von **Object**

in VM implementiert!

Referenzgleichheit !

```
public class Object {

  public native int hashCode();
  public boolean equals(Object obj) { return (this == obj); }
  public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
  }

  public final native Class<?> getClass();
  public final native void notify();
  public final native void notifyAll();
  public final native void wait(long timeout) throws InterruptedException;
  public final void wait(long timeout, int nanos) throws InterruptedException { ... }
  public final void wait() throws InterruptedException { ... }

  protected native Object clone() throws CloneNotSupportedException;

  @Deprecated
  protected void finalize() throws Throwable { }
}
```

- Methoden **toString**, **hashCode**, **equals** können/sollen überschrieben werden
- **getClass**, **wait**, **notify** können nicht überschrieben werden (siehe **final**)
- **clone** muss überschrieben werden, wenn man Kopien erzeugen will
- **finalize** ist "deprecated" und soll nicht mehr verwendet werden

© H. Prähofer, 2026

Überschreiben von Methoden und Super-Aufrufe

- Überschreibende Methoden können mit Super-Aufrufen überschriebene Methode aufrufen

```
public class Timer  {
  protected int hrs, min, sec;
  public Timer(int hrs, int min, int sec) {
    this.hrs = hrs; this.min = min; this.sec = sec;
  }

  public void add(int s) {
       sec += s;
       if (sec > 59) { min += sec/60; sec %= 60; }
       if (min > 59) { hrs += min/60; min %= 60; }
  }
  ...
}
```

```
public class ClockTimer extends Timer {
  public ClockTimer (int h, int m, int s, int max) {
    super(h, m, s);
    maxHrs = max;
  }

  @Override
  public void add(int s) {
    super.add(s);
    hrs = hrs % maxHrs;
  }
  ...
}
```

Geerbte Methode kann **überschrieben** werden, indem man sie mit **derselben Signatur** neu deklariert.

➔ **verdeckt die geerbte Methode!**

Super-Aufruf
überschriebene Methode der Basisklasse kann mit **super** aufgerufen werden

© H. Prähofer, 2026

Annotation @Override

Annnotation @Override zeigt Überschreiben von Methode an

- kann auch ausgelassen werden
- ist aber empfohlen

```
public class Timer  {


  public void add(int s) {
    ...
  }

}
```

```
public class ClockTimer extends Timer {

  @Override
  public void add(int s) {
    …
  }
  …
}
```

- wenn angegeben, dann Überprüfung durch Compiler

```
public class ClockTimer extends Timer {

  @Override
  public void add() {
    …
  }
  …
}
```

Fehler: **add()** überschreibt **add(int)** nicht!

© H. Prähofer, 2026

Sichtbarkeitsattribut protected

protected-Felder/Methoden sind sichtbar:

- in allen **Klassen** des **deklarierenden Pakets**
- in allen **Unterklassen** der Klasse (**auch in anderen Packages**)

- **protected** Felder können in Unterklassen zugegriffen werden
- **protected** Methoden können in Unterklassen aufgerufen werden

```
package timer;
public class Timer  {
  protected int hrs, min, sec;
  private PrintStream logStream = System.out;

  ...
  public void add(int s) {
    ...
  }
  protected void log(String msg) {
    logStream.println(msg);
  }
}
```

```
package timer.clock;
public class ClockTimer extends Timer {
  private int maxHrs;

  public ClockTimer(int h, int m, int s, int max) {
    super(h, m, s);
    maxHrs = max;
  }

  @Override
  public void add(int s) {
    super.add(s);
    hrs = hrs % maxHrs;
    log("Added: " + sec + "secs");
  }

}
```

Beachte: anderes Package!

auf **protected** Felder kann man zugreifen

protected Methoden können aufgerufen werden

© H. Prähofer, 2026

Sichtbarkeitsattribut protected

protected-Felder/Methoden sind sichtbar:

- in allen **Klassen** des **deklarierenden Pakets**
- in allen **Unterklassen** der Klasse (**auch in anderen Packages**)

- **protected** Felder können in Unterklassen zugegriffen werden
- **protected** Methoden können in Unterklassen aufgerufen werden
- **protected** Methoden können in Unterklassen überschrieben werden

```
package timer;
public class Timer  {
  protected int hrs, min, sec;
  private PrintStream logStream = System.out;

  ...
  public void add(int s) {
    ...
  }
  protected void log(String msg) {
    logStream.println(msg);
  }
}
```

```
package timer.clock;
public class ClockTimer extends Timer {
  private int maxHrs;

  public ClockTimer(int h, int m, int s, int max) {
    super(h, m, s);
    maxHrs = max;
  }
  @Override
  public void add(int s) {
    super.add(s);
    hrs = hrs % maxHrs;
    log("Added: " + sec + "secs");
  }
  @Override
  protected void log(String msg) {
    super.log(msg);
    ...
  }
}
```

Beachte: anderes Package!

protected Methoden können überschrieben werden

© H. Prähofer, 2026

Erweiterung der Sichtbarkeit

Sichtbarkeit kann in überschreibenden Methode erweitert werden

- damit erweitert man die Schnittstelle in der Subklasse

Zum Beispiel von protected zu public

- **getId() protected** in **Person**
- **getId() public** in Subklasse **Student**

```
public class Person  {
  private int id;
  private String name;
  ...

  protected int getId() {
    return id;
  }
}
```

Sichtbarkeit erweitert

```
public class Student extends Person {

  ...

  @Override
  public int getId() {
    return super.getId();
  }
}
```

```
Person person = new Person("Hans");
int id = person.getId();
```

✗

nicht öffentlich sichtbar für **Person**

```
Student student = new Student("Hans");
int id = student.getId();
```

aber öffentlich sichtbar für **Student**

© H. Pfahler, 2020

Abstrakte Klassen und Methoden

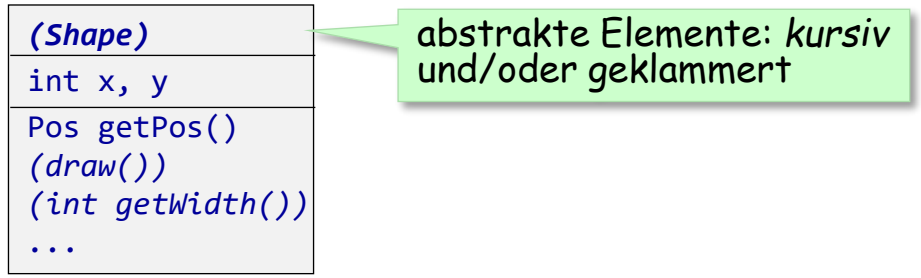
```
public abstract class Shape {
  int x, y;

  protected Shape(int x, int y) { this.x = x; this.y = y; }

  public Pos getPos() { return new Pos(x, y); }

  public abstract void draw() ;
  public abstract int getWidth() ;
  ...
}
```

Klassendiagramm



- Eine **Methode ist abstrakt**, wenn sie **keine Implementierung** hat
  - sie muss mit dem Schlüsselwort **abstract** gekennzeichnet werden
  - wird mit **einem Strichpunkt abgeschlossen**
  - definiert das **„Vorhandensein“** einer Methode, lässt aber konkrete **Implementierung offen**
- Eine **Klasse**, die **mindestens 1 abstrakte Methode** hat, **muss selbst abstrakt** sein
  - Aber: eine Klasse mit **nur konkreten Methoden** kann auch **abstrakt** sein
- Von **abstrakten Klassen** können **keine Objekte** erzeugt werden

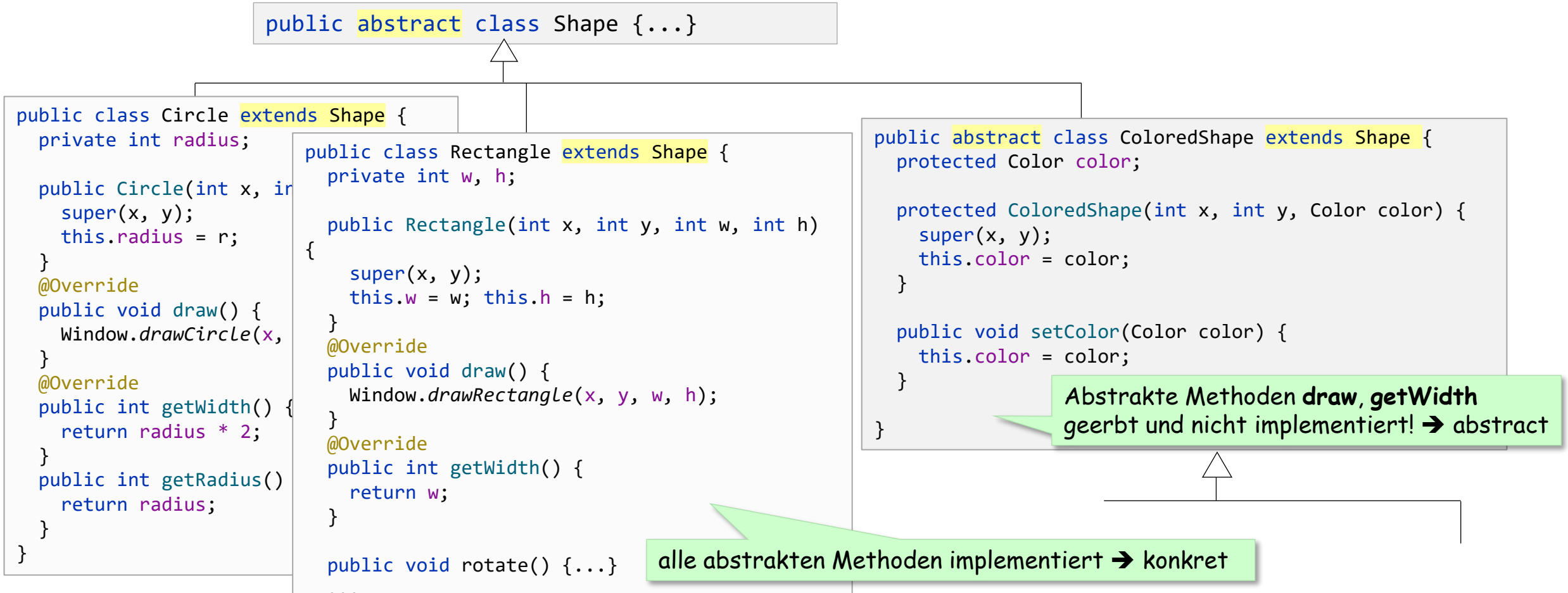
```
Shape shape = new Shape(0, 0);
```
✗ Compilerfehler

© H. Prähofer, 2026

Ableitung von Abstrakter Klasse

Unterklassen von abstrakten Klassen

- konkrete Klassen **müssen geerbte abstrakte Methoden implementieren**
- **oder** sie müssen **selbst abstrakt** sein



© H. Prähofer, 2026

Abstrakte Klassen als Typen

- Eine **abstrakte Klasse** kann **als Typ** in Deklarationen verwendet werden:

```
Shape shape = new Circle(100, 100, 10);
```

```
Shape[] shapes = new Shape[2];
shapes[0] = new Rectangle(10, 20, 30, 10);
shapes[1] = new Circle(10, 20, 30);
```

- Die abstrakte Klasse bestimmt, **welche Methoden vorhanden** sind
  - Damit kann man auch **alle Methoden der abstrakten Klasse aufrufen** (auch abstrakte)

```
shape.draw();
int w = shape.getWidth();
```

```
for (Shape shape : shapes) {
  shape.draw();
}
```

draw, **getWidth** in **Shape** abstrakt, aber in Klassen **Rectangle** und **Circle** konkret

© H. Prähofer, 2026

final Klassen

Von **Klassen**, die als **final** deklariert sind, dürfen **keine Unterklassen** abgeleitet werden

➔ Sperren von Bilden von Subklassen

```
public final class Account {
  int balance;
  public void deposit (int value) {
    balance = balance + value;
  }
  public void withdraw (int value) {
    balance = balance - value;
  }
}
```

Beachte: **final** Klassen können nicht **abstract** sein!

```
public class FakeAccount extends Account {

  public void deposit (int value) {
    balance = balance + value / 2;
  }
}
```

✗ final Klasse Account kann nicht erweitert werden

Wegen dynamischer Bindung von

```
account.deposit(value);
```

könnte sonst die Semantik dieser Operation geändert werden.

Anwendung: Für Klassen, deren Semantik **unabänderlich** sein soll (z.B. **String**)

© H. Prähofer, 2026

final Methoden

Methoden, die als **final** deklariert sind, dürfen **in Unterklassen nicht überschrieben** werden

➔ **Schutz vor Überschreiben**

```
public class Account {
  protected int balance;

  public final void deposit(int value) {
    balance = balance + value;
  }
  public final void withdraw(int value) {
    balance = balance - value;
  }
  public void print() {
    System.out.println(balance);
  }
}
```

```
public class MyAccount extends Account {

  @Override
  public void print () {
    System.out.println(balance + ",00");
  }

  public void deposit(int value) { }
  public void withdraw(int value) { }
}
```

✗ final Methoden können nicht überschrieben werden

Bilden einer Unterklasse ist erlaubt:

- **print** darf auch überschrieben werden,
- **deposit** und **withdraw** aber nicht

© H. Prähofer, 2026

Überdecken von Feldern

Unterklasse könn(t)en Datenfelder in der Oberklasse überecken

```
class A {
  int x;
  A() {
    x = 1;
  }
}
```

Typen überdeckender Felder können verschieden sein

```
class B extends A {
  String x;
  B() {
    x = "abc";
    super.x = 3;
  }
}
```

überdeckt x aus A

Zugriff auf überdecktes Feld mit **super**

```
A a = new A();
int xA = a.x;
```

```
B b = new B();
String xB = b.x;
```

Aber beachte:

```
A a = new B();
int x = a.x;
```

ist **x** von **A**

Felder sind statisch gebunden!

Regel: Überdecken von Feldern vermeiden!

- kaum nützlich und Fehlerquelle
- besser anderen Name wählen

© H. Prähofer, 2026

Lernziele überprüfen

- Wie wird in Java eine Unterklasse von einer Superklasse gebildet?
- Was bedeutet Überschreiben von Methoden?
- Wozu dient die Annotation @Override und warum sollte man diese angeben?
- Welche Methoden der Klasse Object können/sollen in Unterklassen überschrieben werden?
- Warum muss im Konstruktor einer Unterklasse ein Konstruktor der Oberklasse aufgerufen werden?
- Wie kann man einen Super-Aufruf in einer Methode einsetzen?
- Wie definiert man eine abstrakte Methode?
- Was ist eine abstrakte Klasse, welche Eigenschaften hat sie und in welchen Fällen setzt man sie ein?
- Welche Bedeutung hat das Sichtbarkeitsattribut protected im Zusammenhang mit Vererbung?
- Warum kann man die Sichtbarkeit von Methoden in Unterklassen erweitern?
- Welche Wirkung hat das Schlüsselwort final bei Klassen und bei Methoden im Zusammenhang mit Vererbung?
- Warum soll man Felder nicht überschreiben?

© H. Prähofer, 2026

2. Vererbung und Dynamische Bindung

2.1 Vererbung bei Klassen

2.2 Typhierarchien und dynamische Bindung

2.3 Kovarianz bei Rückgabewerten und Exceptions

2.4 Fallstudie List

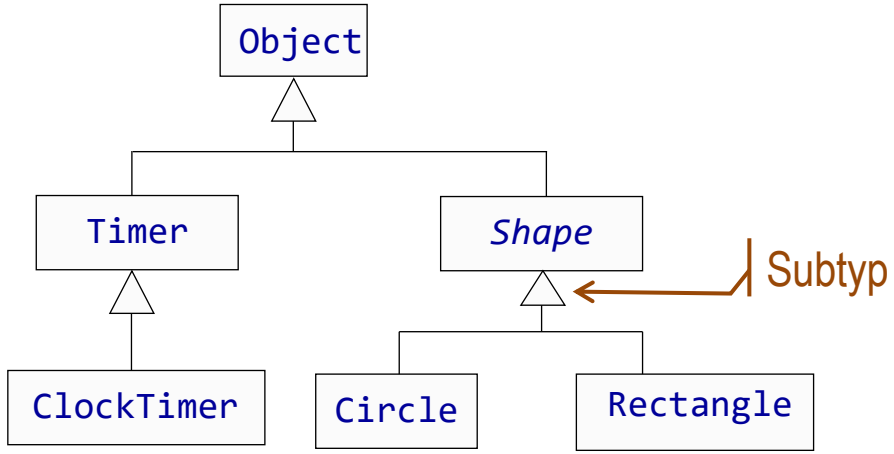
2.5 Entwurfsmuster

© H. Prähofer, 2026

Typen, Subtypen und Typhierarchien

Typ

- eine **Klasse** führt einen **Typ** ein
- Typ wird für die **Deklaration von Variablen**, **Parameter** etc. verwendet
- Der Typ bestimmt, **welche Operationen verfügbar** sind

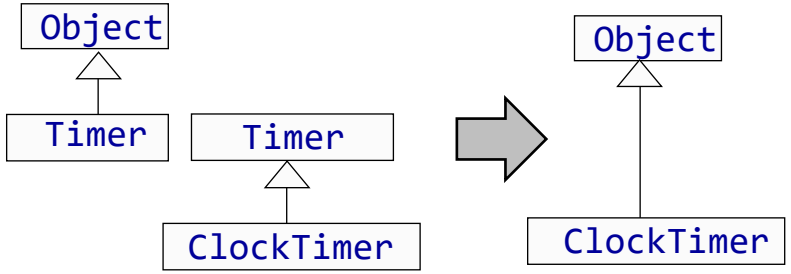


Subtyp

- Mit **Vererbung** wird eine **Subtyp-Beziehung** eingeführt
- Ein **Subtyp** ist mit seinem **Supertyp kompatibel**

Typhierarchie

- **Subtyp-Beziehung** ist **transitiv**
- Mit einer **Klassenhierarchie** ist eine **Typhierarchie** verbunden



© H. Prähofer, 2026

Typkompatibilität und polymorphe Zuweisung

Typkompatibilität

- **Subklasse** ist mit **Superklasse kompatibel**

Vom Compiler geprüft!

Eine Subklasse muss die Schnittstelle der Superklasse mindestens erfüllen

- man kann auf **alles zugreifen**, was **Superklasse erlaubt**
- Überschreibende Methoden müssen bezüglich **Rückgabetyp** und geworfenen **Exceptions kompatibel** sein

siehe Abschnitt zu Kovarianz bei Rückgabewerten und Exceptions

Polymorphe Zuweisung

- **Typkompatibilität** ermöglicht eine **Subklasse immer dort einzusetzen**, wo **Superklasse verlangt wird**
- Einer Variablen eines **Supertyps** kann ein **Objekt eines Subtyps** zugewiesen werden
- **Subtyp-Beziehung** ist **transitiv**

Supertyp

```
Shape shape;

shape = new Circle(10, 10, 5);
shape = new Rectangle(10, 10, 20, 10);
```

Subtyp

```
Object object;

object = new Circle(10, 10, 5);
object = new Rectangle(10, 10, 20, 10);
```

© H. Prähofer, 2026

Statischer und dynamischer Typ

```
Shape shape = new Circle(10, 10, 5);
```



- Typ, mit dem die **Variable deklariert** ist
- Der statische Typ ist **fix** wie bei der **Deklaration angegeben** (*statisch*)

```
Shape shape;
```

- Bestimmt, **welche Methoden aufrufbar** sind
- Wird **vom Compiler** geprüft

- Typ des **Objekts**, auf das die Variable verweist
- Der dynamische Typ kann sich durch Zuweisung anderer Objekte **ändern** (*dynamisch*)

```
shape = new Circle(10, 10, 5);
shape = new Rectangle(5, 10, 20, 30);
```

- Bestimmt, **welche überschriebene Methode ausgeführt** wird
- Wird **zur Laufzeit** wirksam

Dynamische Typ muss **Subtyp oder gleich** dem **statischem Typ** sein!

© H. Prähofer, 2026

Dynamische Bindung von Methoden

```
Shape shape = new Circle(10, 10, 5);
```

Statischer Typ

Dynamischer Typ

- **Statischer Typ** bestimmt welche Methoden **sichtbar** sind (definiert die ***Schnittstelle***)
- **Dynamischer Typ** bestimmt, welche Methoden **ausgeführt** werden (bestimmt ***Implementierung***)

```
shape.draw();

int w = shape.getWidth();
```

statisch sichtbare Methoden

```
abstract class Shape {
  public abstract void draw();
  public abstract int getWith() {...}
  …
}
```

statischer Typ

dynamischer Typ

```
public class Circle extends Shape {
  ...
  public int getRadius() {...}

  @Override
  public void draw() {...}

  @Override
  public int getWidth() {...}
  ...
}
```

dynamisch gebundene Methoden

```
shape.getRadius();
```

✗

statisch nicht sichtbar

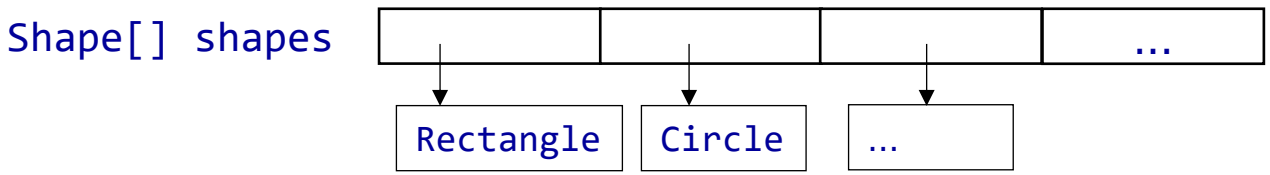
Polymorphe Verarbeitung

- **Unterschiedliche Objekte** können auf **einheitliche Weise** verarbeitet werden

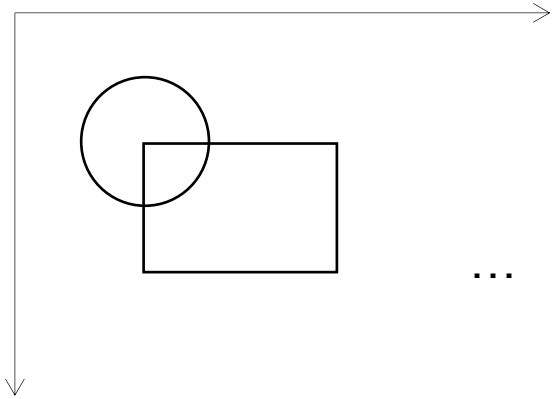
Beispiel: Array mit Objekten unterschiedlicher **Shape**-Unterklassen

```
Shape[] shapes = new Shape[10];
shapes[0] = new Rectangle(20, 20, 30, 20);
shapes[1] = new Circle(20, 20, 10);
...
```

Polymorphe Zuweisungen




```
for (Shape shape: shapes) {
    shape.draw();
}
```



© H. Prähofer, 2026

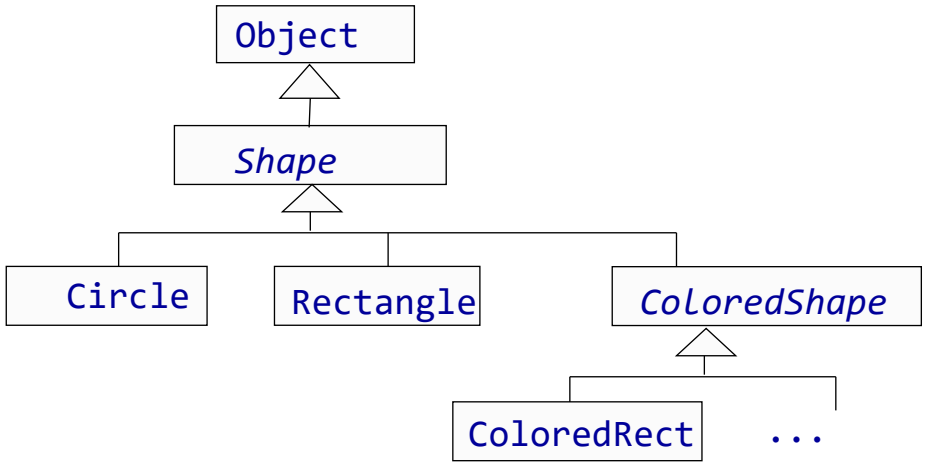
Laufzeit-Typtest

```
Shape shape = …
```

Prüfen des Typ des Objekts zur Laufzeit

```
if (shape instanceof Circle) ...
```

true, wenn **shape** vom dynamischen Typ **Circle** (oder einer **Subklasse** davon) ist.



Beispiele

```
Shape shape = new Circle(10, 10, 20);
```

```
if (shape instanceof Object)...
if (shape instanceof Shape)...
if (shape instanceof Circle) ...
if (shape instanceof Rectangle)...
if (shape instanceof ColoredShape)
...
```

```
true
true
true
false
false
```

© H. Prähofer, 2026

Typumwandlung (Typecast)

Mit einem Typecast kann die statische Sicht auf einen Wert geändert werden

```
Shape shape = new Circle(5, 5, 10);
```

```
  (Circle) shape
```

Sicht des Wertes von **shape** wird auf **Circle** erweitert !

- damit kann der Wert als **Circle** verwendet werden

```
((Circle) shape).getRadius();
```

Bedingungen

- Zieltyp des Casts muss zum **statischen Typ kompatibel** sein ➔ sonst **Compiler-Fehler**

```
... (Timer) shape ...
```

✗ Compiler-Fehler: **Shape** inkompatibel mit Typ **Timer**

- Objekt zur Laufzeit muss vom Zieltyp oder Subtyp sein ➔ sonst **ClassCastException**

```
... (Rectangle) shape ...
```

↯ Laufzeitfehler: **ClassCastException**, Wert ist nicht vom Typ **Rectangle** (sondern Circle)

© H. Prähofer, 2026

Kombination Typtest und Typecast

- Sicherer Zugriff: Typtest gefolgt von Typecast

```
if (shape instanceof Rectangle) {
  Rectangle rect = (Rectangle) shape;
  rect.rotate();
}
```

häufiger Anwendungsfall

- Neuer instanceof-Test mit "Pattern Matching"
  - kombiniert Typtest, Typecast und Variablenbindung

seit Java 16

Typtest auf **Rectangle**

Variablenbindung an "Pattern-Variable" vom Typ **Rectangle**

```
if (shape instanceof Rectangle rect) {
  rect.rotate();
}
```

© H. Prähofer, 2026

Exkurs: Switch-Ausdruck

Switch-Ausdruck unterscheidet Fälle und definiert **Wert abhängig von Fällen**

Syntax ähnlich Switch-Anweisung aber

- ist **Ausdruck** und **liefert** damit einen **Wert**
- Wert für jeweiligen Fall bestimmt durch Ausdruck nach Pfeil `->`
- nach **case mehrere Fälle** durch Beistrich getrennt
- kein **break**
- **default** für Wert, wenn keiner der Fälle zutrifft

notwendig, wenn Werte in Cases nicht vollständig abgedeckt

```
int month = …;

int days =
  switch (month) {
    case 1, 3, 5, 7, 8, 10, 12 -> 31;
    case 4, 6, 9, 11 -> 30;
    case 2 -> 28;
  };
```

Ausdruck für Fallunterscheidung

Rückgabewert des Switch-Ausdrucks

für Fall gelieferter Wert

© H. Prähofer, 2026

Exkurs: Switch-Ausdruck

yield

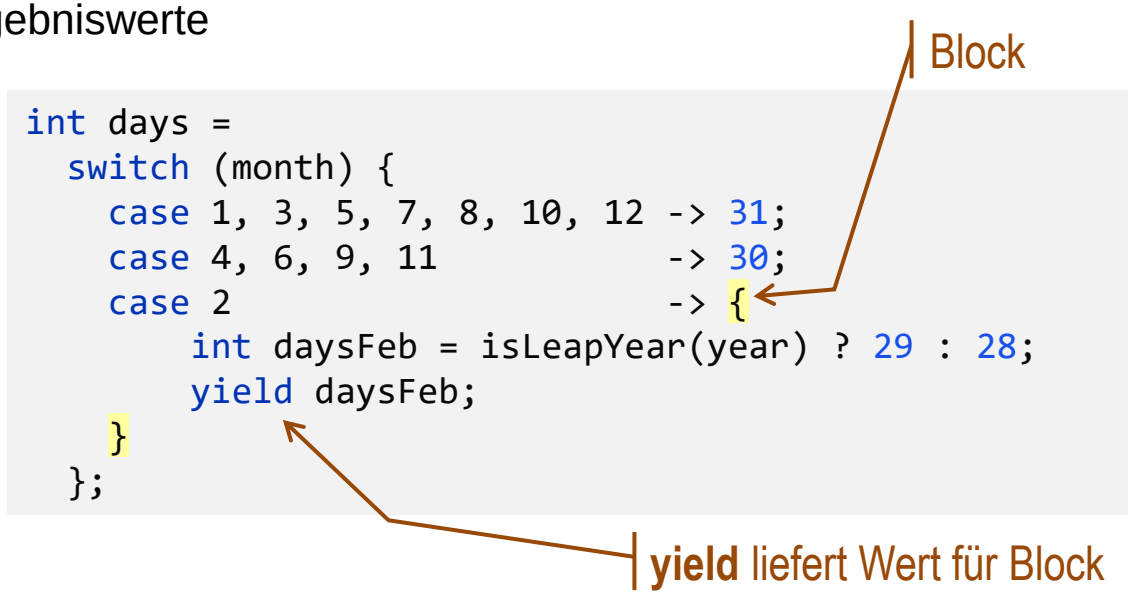
- Statt `->` kann `: yield` verwendet werden

```
int days =
  switch (month) {
    case 1, 3, 5, 7, 8, 10, 12  : yield 31;
    case 4, 6, 9, 11            : yield 30;
    case 2                      : yield 28;
};
```

- **yield** erlaubt mehrere Anweisungen zur Berechnung der Ergebniswerte

```
int days =
  switch (month) {
    case 1, 3, 5, 7, 8, 10, 12 : yield 31;
    case 4, 6, 9, 11           : yield 30;
    case 2                     :
        if (isLeapYear(year))   yield 29;
        else                    yield 28;
  };
```

```
int days =
  switch (month) {
    case 1, 3, 5, 7, 8, 10, 12 -> 31;
    case 4, 6, 9, 11           -> 30;
    case 2                     -> {
        int daysFeb = isLeapYear(year) ? 29 : 28;
        yield daysFeb;
    }
  };
```



© H. Prähofer, 2026

Switch mit Type Patterns

Typprüfung in Switch

seit Java 21

- Unterscheidung der Fälle mit Typen (wie bei **instanceof**-Prüfung)
- Zuweisen des Objekts an Pattern-Variable mit speziellem Typ (= Typcast)
- Ausführen des Falls unter Verwendung der Pattern-Variablen (mit speziellem Typ)

```
Shape shape = ...;

double area =
    switch (shape) {
      case Rectangle rect -> rect.getWidth() * rect.getHeight();
      case Circle circle  -> circle.getRadius() * circle.getRadius() * Math.PI;
      default             -> throw new IllegalArgumentException("Unrecognized shape");
    };
```

Pattern-Variablen in Scope

Fälle mit Typtest und Binden an Pattern-Variablen

© H. Prähofer, 2026

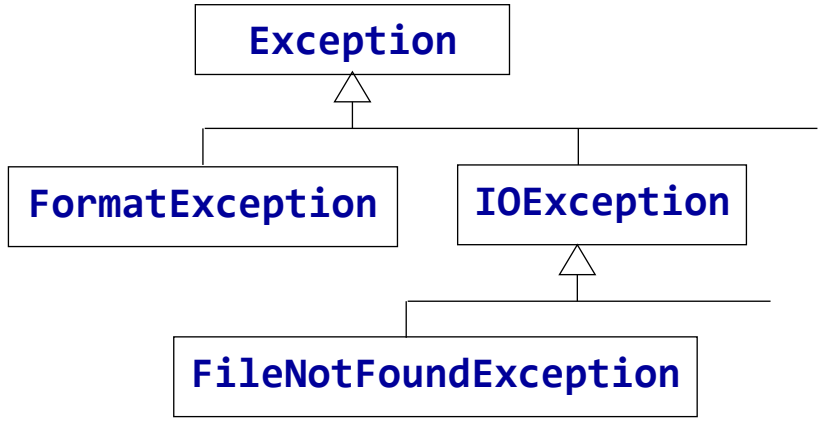
Ausnahmebehandlung: Fangen von Exceptions

Fangen von Exceptions

- Unterscheidung der Exception-Klassen in **catch**-Klauseln
- Auswahl der **catch-Klausel** durch **Typtest** des **dynamischen Typs** der geworfenen Exception

wie **instanceof**

- Mehrere catch-Klauseln: **zuerst speziellere**, **dann allgemeinere** Exception-Typen




```
Reader reader = null;
try {
  reader = new FileReader(fileName);
  int x = readInt(new FileReader(fileName);

} catch (FormatException formatException) {
  ...
} catch (FileNotFoundException fileNotFoundException) {
  ...
} catch (IOException ioException) {
  ...
} catch (Exception exception) {
  ...
} finally { … }
```

fängt **FormatExceptions**

fängt **FileNotFoundExceptions**

fängt alle **IOExceptions**

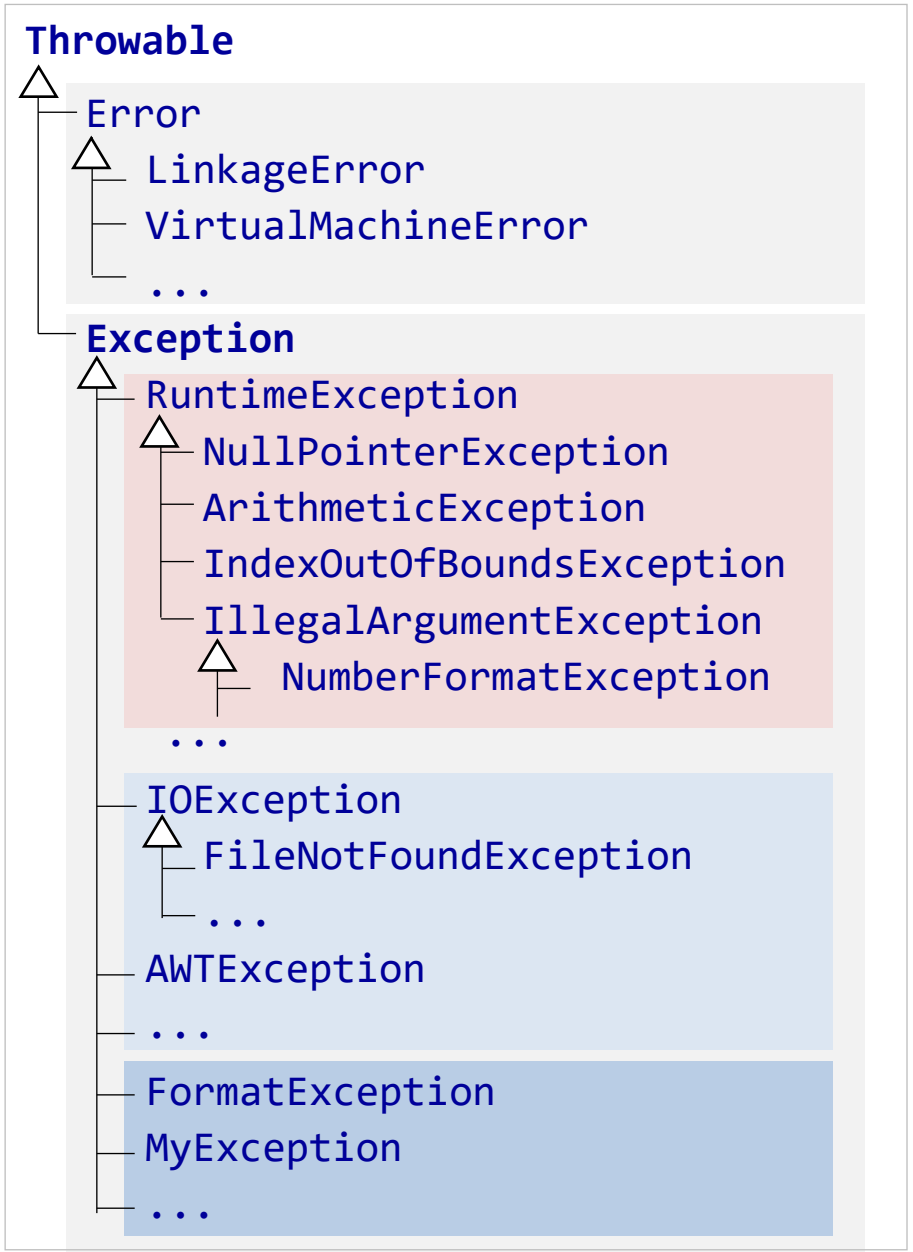
fängt alle restlichen **Exceptions**

spezieller

allgemeiner

© H. Prähofer, 2026

Hierarchie der Exception-klassen



schwere Fehler der Laufzeitumgebung
(müssen nicht behandelt werden)

Ausnahmen für Programmfehler
(müssen nicht behandelt werden)

Runtime Exceptions

vordefinierte Ausnahmen für Ausnahmesituationen
(müssen behandelt werden)

selbst definierte Ausnahmen
(müssen behandelt werden)

Checked Exceptions

© H. Prähofer, 2026

throws-Klausel bei Methoden

- Eine Methode muss Checked Exceptions behandeln oder deklarieren
  - entweder behandeln **durch Fangen** innerhalb der Methode (wie gesehen)
  - oder mit **throws-Klausel** mögliches Werfen deklarieren

```
package java.io;
public abstract class Reader {
  public abstract char read() throws IOException { … }
  ...
}
```

read deklariert **IOException**

- Ruft man eine Methode mit einer throws-Klausel auf
  - muss man entweder diese fangen

```
static String readDigits(Reader reader) {
  try {
    char d = reader.read();
    ...
  } catch (IOException e) {...}
}
```

kann **IOException** fangen

  - oder die Methode muss mit einer **throws-Klausel** die **Exception weitergeben**

```
static String readDigits(Reader reader) throws IOException {
  char d = reader.read();
  ...
}
```

read deklariert **IOException**

© H. Prähofer, 2026

throws-Klausel bei Methoden

- Mehrere Exception-Typen in einer throws-Klausel

```
static String readDigits(Reader reader) throws IOException {
  char d = reader.read();
  ...
}
```

```
static int readInt(Reader reader) throws IOException, FormatException {
  String ds = readDigits(reader);
  if (ds.length() == 0) {
    throw new FormatException(ds);
  }
  …
}
```

muss **IOException** und **FormatException** deklarieren

kann **IOException und FormatException** werfen

© H. Prähofer, 2026

Merksätze

- In Java sind Methoden dynamisch gebunden
  - D.h., es wird die überschreibende Methode aufgerufen
- Auch ein Aufruf im Konstruktor ist dynamisch gebunden und verzweigt zur überschreibenden Methode
  - Damit sind Aufrufe von dynamisch gebundenen Methoden im Konstruktor gefährlich, weil dadurch vor der finalen Initialisierung zur Methoden der Unterklasse verzweigt wird
- Methoden können nur überschrieben werden, wenn sie in der Unterklasse sichtbar sind
  - Damit können private Methoden nicht überschrieben werden
  - Ebenso können package-private durch Methoden aus Unterklassen in anderen Packages nicht überschrieben werden
- Als Template-Methoden bezeichnet man Methoden in Oberklassen, die Aufrufe zu speziellen Methoden in Unterklassen enthalten
  - Sie sind ein wesentliches Entwurfsprinzip bei der objektorientieren Programmierung
- Überladen von Methoden passiert auf Basis des statischen Typs der Parameter
  - Damit sind überladene Methoden in denen die Parametertypen in einer Subtyp-Beziehung stehen zu vermeiden

© H. Prähofer, 2026

Lernziele überprüfen

- Was bedeutet Typkompatibilität?
- Was versteht man unter polymorpher Zuweisung?
- Warum ermöglicht Typkompatibilität polymorphe Verarbeitung?
- Was ist der Unterschied zwischen statischem Typ und dynamischem Typ?
- Wie funktioniert die dynamische Bindung von Methoden in Java?
- Welche Rolle spielt der statische Typ, welche der dynamische Typ bei einem Methodenaufruf?
- Wozu dient der Laufzeit-Typtest mit instanceof, und wie kann er mit einem Typecast kombiniert werden?
- Prüft ein instanceof-Test den statischen oder den dynamischen Typ?
- Was bewirkt ein Typecast?
- In welchen Fällen führt ein Typecast zu einem Compiler-Fehler, in welchen zu einem Laufzeitfehler?
- Was ist ein instanceof-Test mit Pattern Matching, wie wird er geschrieben, und was bewirkt er?
- Wie schreibt man ein switch mit Type Patterns?

© H. Prähofer, 2026

2. Vererbung und Dynamische Bindung

2.1 Vererbung bei Klassen

2.2 Typhierarchien und dynamische Bindung

2.3 Kovarianz bei Rückgabewerten und Exceptions

2.4 Fallstudie List

2.5 Entwurfsmuster

© H. Prähofer, 2026

Kovariante Rückgabetypen

- Rückgabetypen **überschreibender Methoden** können **spezieller** werden
- Man spricht von **kovarianten** Rückgabetypen

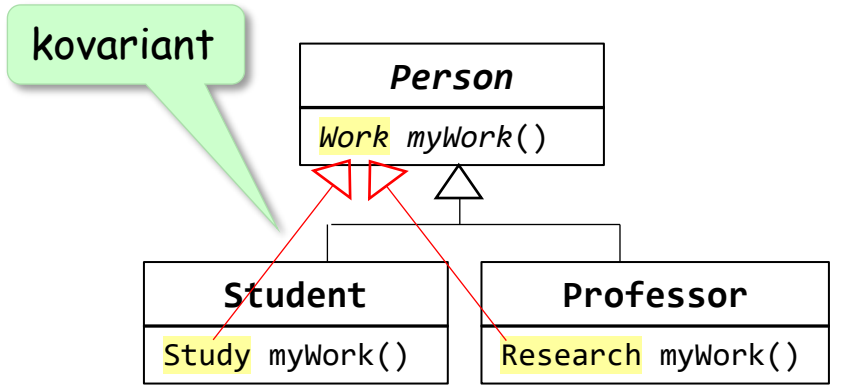
in spezielleren Klasse ist auch Rückgabetyp spezieller

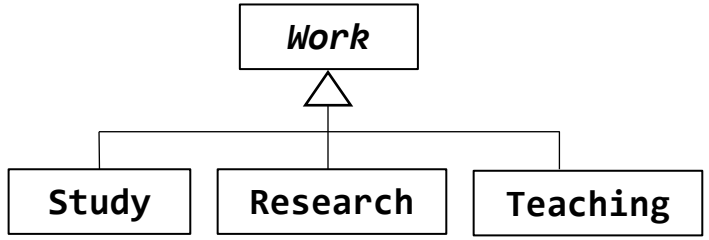
```
public abstract class Person {
  ...
  public abstract Work myWork();
}
```

```
public class Student extends Person {
  private final Study study;
  ...
  @Override
  public Study myWork() {
    return study;
  }
}
```

Student spezieller als **Person**
Study spezieller als **Work**

```
public class Professor extends Person {
  private final Research research;
  ...
  @Override
  public Research myWork() {
    return research;
  }
}
```





© H. Prähofer, 2026

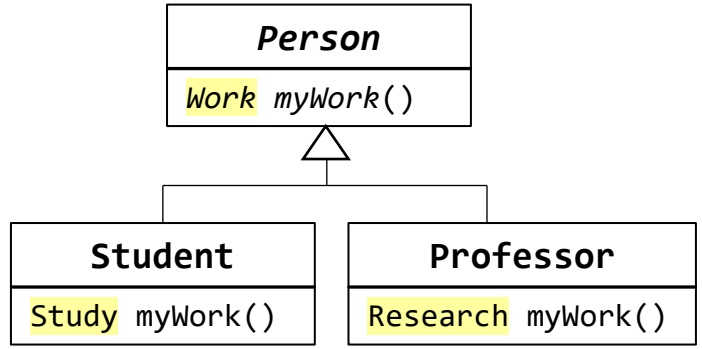
Kovariante Rückgabewerte und Polymorphe Zuweisungen

■ **Kovariante Rückgabetypen** erhalten die **Typkompatibilität** bei polymorphen Zuweisungen

Polymorphe Zuweisungen:

```
Person person = ... ;
Work work = person.myWork();
```

Person definiert als Rückgabetyp **Work**



☐ Student

```
Person person = new Student("Hans", new Study("CS"));
Work work = person.myWork();
```

Student retourniert **Study**
kompatibel mit **Work**

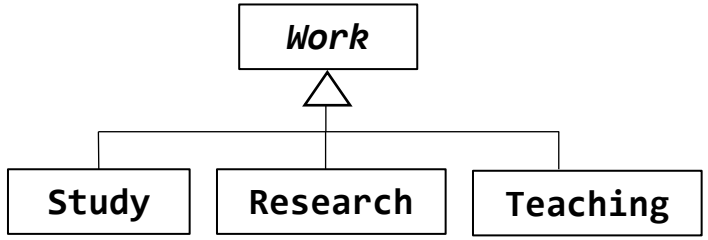
```
Work work = { …  return study; }
```

☐ Professor

```
Person person = new Professor("Wirth", new Research("Compiler"));
Work work = person.myWork();
```

Professor retourniert **Research**
kompatibel mit **Work**

```
Work work = { …  return research; }
```



© H. Prähofer, 2026

Kovarianz bei geworfenen Exceptions

Exceptions in **throws-Klauseln** in **überschreibenden Methoden** können **spezieller** sein bzw. können **weniger Exceptions** sein

```
public abstract class Person {
  ...
  public abstract void doWork(Work w) throws WorkException;
}
```

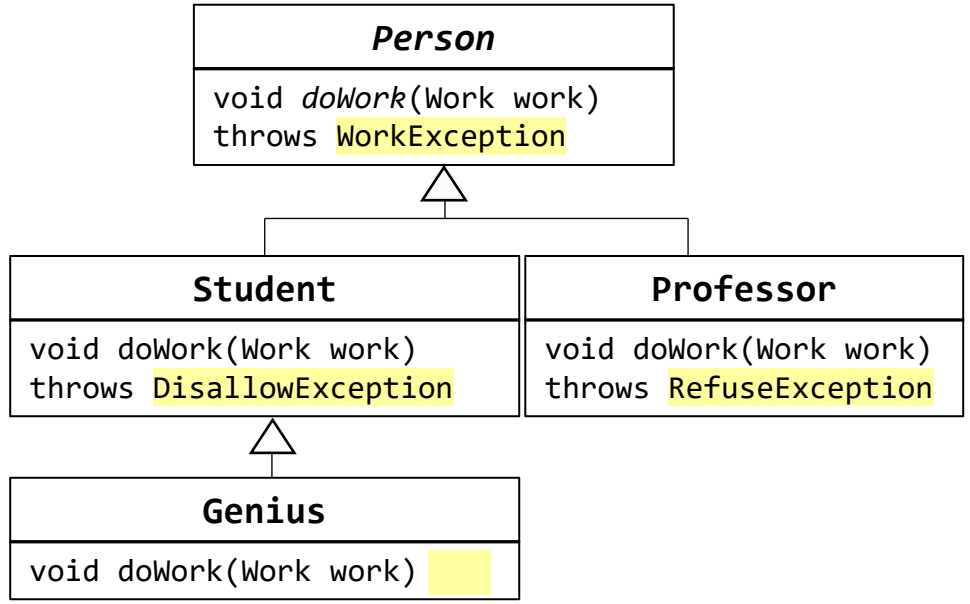
```
public class Professor extends Person {
  ...
 public void doWork(Work w) throws RefuseException {
   if (w instanceof Study) throw new RefuseException(w);
   ...
  }
}
```

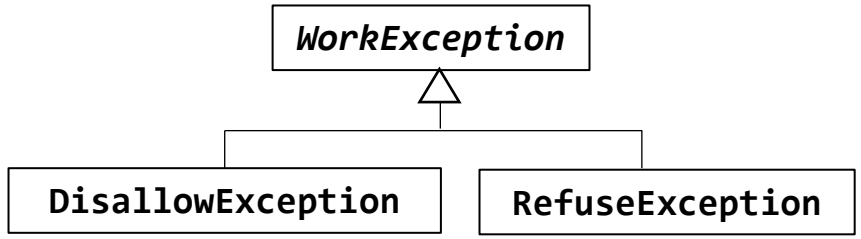
speziellere Exception

```
public class Student extends Person {
  ...
  public void doWork(Work w) throws DisallowException {
    if (w instanceof Teaching) throw new DisallowException(w);
    ...
  }
}
```

```
public class Genius extends Student {
  ...
  public void doWork(Work w)  { ... }
}
```

keine Exception





© H. Prähofer, 2026

Kovarianz bei geworfenen Exceptions

Exceptions in **throws-Klauseln** in **überschreibenden Methoden** können **spezieller** sein bzw. können **weniger Exceptions** sein

```
Person person = …  ;
try { person.doWork(work); }
catch (WorkException e) {...}
```

Person deklariert **WorkException**
WorkException muss gefangen werden

- **Student**

```
Person person = new Student("Hans", new Study("CS"));
try { person.doWork(work); }
catch (WorkException e) {...}
```

```
… throw new DiallowException(w);
```

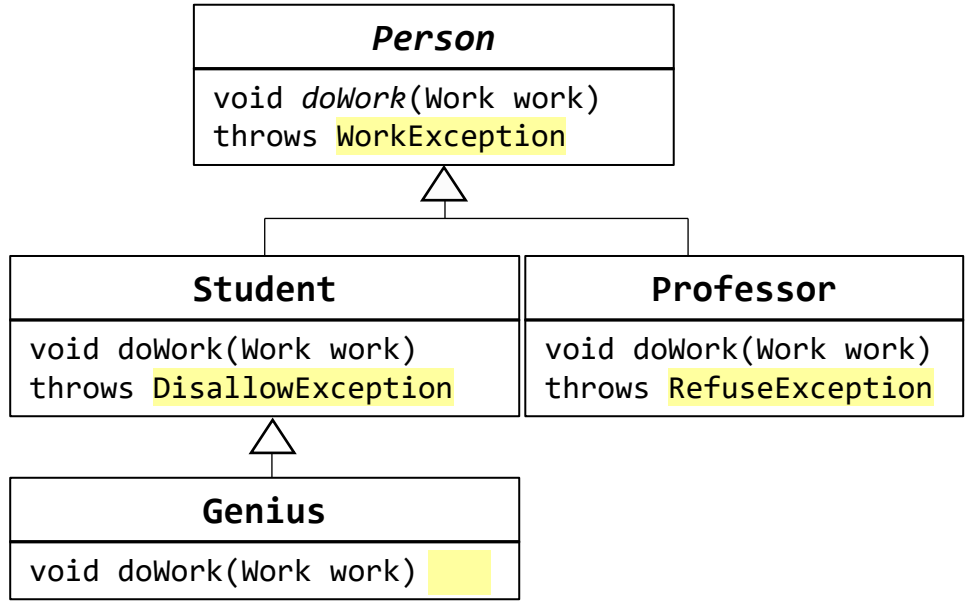
Student wirft **DisallowException**
kompatibel mit **WorkException**

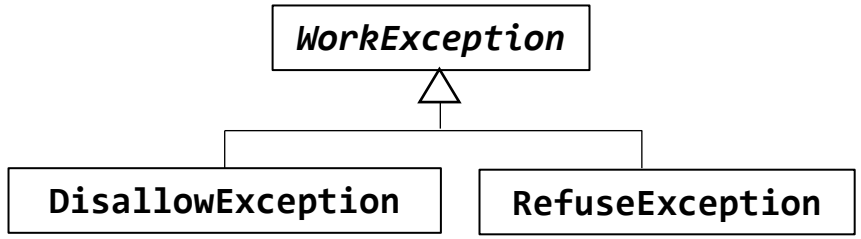
- **Professor**

```
Person person = new Professor("Wirth", new Research("Compiler"));
try { person.doWork(work); }
catch (WorkException e) {...}
```

```
… throw new RefuseException(w);
```

Professor wirft **RefuseException**
kompatibel mit **WorkException**





© H. Prähofer, 2026

Kovarianz bei geworfenen Exceptions

Exceptions in **throws-Klauseln** in **überschreibenden Methoden** können **spezieller** sein bzw. können **weniger Exceptions** sein

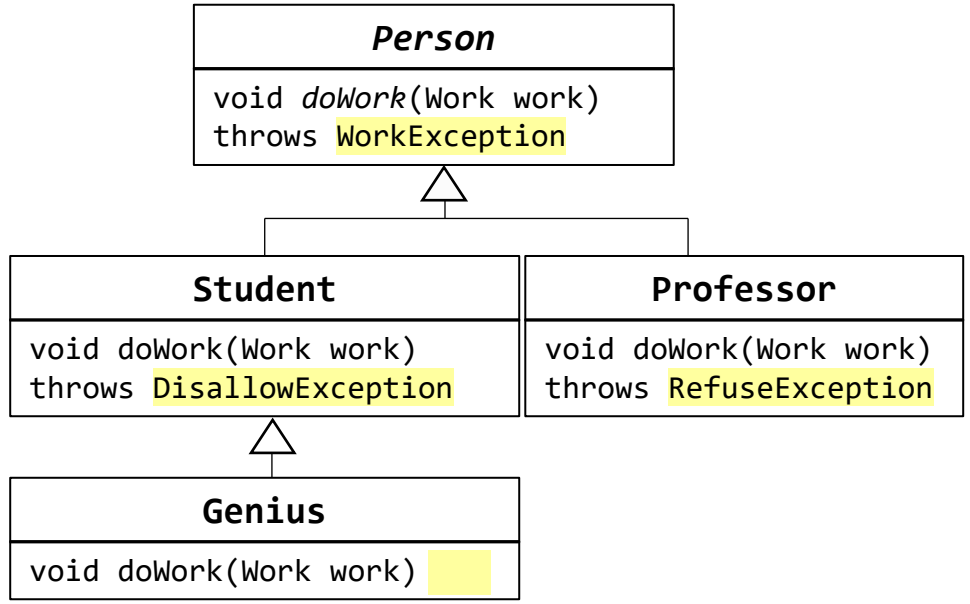
```
Person person = …  ;
try { person.doWork(work); }
catch (WorkException e) {...}
```

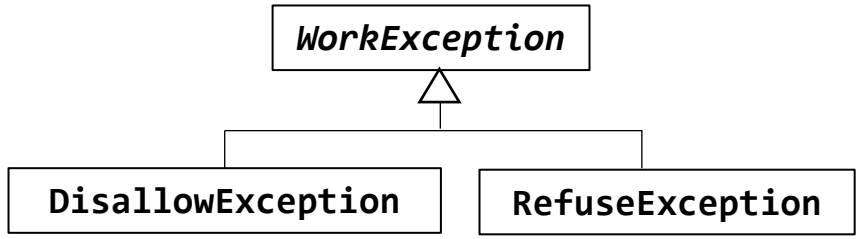
Person deklariert **WorkException**
WorkException muss gefangen werden

- **Genius**

```
Person person = new Genius("Fritz");
try { person.doWork(work); }
catch (WorkException e) {...}
```

Genius wirft keine Exception
damit auch keine Zuweisung





© H. Prähofer, 2026

Lernziele überprüfen

- Was sind kovariante Rückgabetypen?
- Warum sind kovariante Rückgabetypen bei überschreibenden Methoden sinnvoll?
- Welche Regel gilt für Rückgabetypen bei überschreibenden Methoden?
- Welche Regel gilt für deklarierten Checked Exceptions bei überschreibenden Methoden?
- Warum dürfen überschreibende Methoden nur dieselben, speziellere oder weniger Checked Exceptions deklarieren?
- Warum bleibt dadurch die Typkompatibilität bei polymorphen Zuweisungen und Methodenaufrufen erhalten?

© H. Prähofer, 2026

2. Vererbung und Dynamische Bindung

2.1 Vererbung bei Klassen

2.2 Typhierarchien und dynamische Bindung

2.3 Kovarianz bei Rückgabewerten und Exceptions

2.4 Fallstudie List

2.5 Entwurfsmuster

© H. Prähofer, 2026

Fallstudie List

Zielsetzung

- Entwurfsprinzipien für objektorientierte Systeme diskutieren
- dabei insbesondere Vererbungsstrukturen und Schnittstellen einsetzen
- und dadurch auch Gestaltung der Collection-Bibliothek von Java motivieren

Vorgehen

- mehrere Varianten und Entwicklungsstufen mit schrittweisen Verbesserungen
- soll zeigen wie Sprachelemente, vorteilhaft eingesetzt werden können
- Details der Implementierung nicht immer wichtig

Literatur

- viele Entwurfsprinzipien sind dem Buch "Effective Java" entnommen

Joshua Bloch: Effective Java – Best practices for the Java Platform, 3rd edition
Addison-Wesley Professional, 2017

Joshua Bloch verantwortlich für Java Collection-Bibliothek

© H. Prähofer, 2026

Fallstudie List

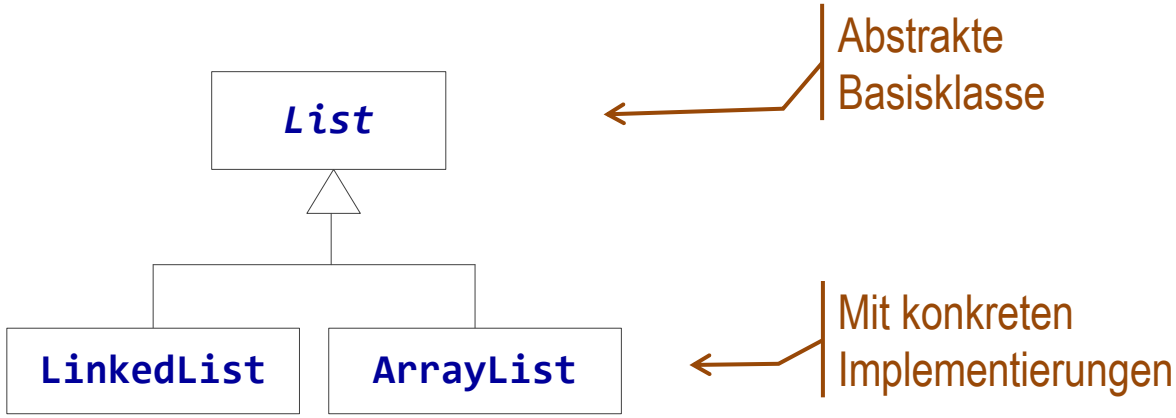
Aufgabenstellung

- Implementierung von Listen mit Objektwerten
- mit Varianten in Implementierung: **LinkedList** und **ArrayList**
- mit Zusatzfunktionen, wie z.B. Logging
- mit Einbindung von ähnlichen Strukturen, z.B. Arrays
- Verwendung von Vererbung, Interfaces und schließlich „Entwurfsmuster“

© H. Prähofer, 2026

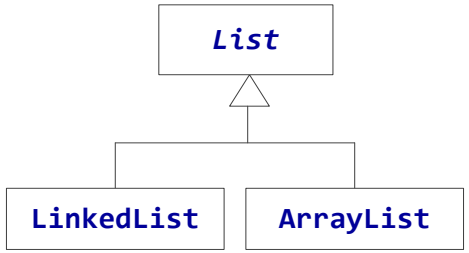
Interface List und Implementierende Klassen

- Abstrakte Basisklasse **List**
  - fixiert Schnittstelle
  - wird als Typ verwendet
  - für Gemeinsamkeiten in Implementierung
- Konkrete Klassen
  - **LinkedList**
  - **ArrayList**
  - ...



© H. Prähofer, 2026

Basisklasse List



Öffentliche Schnittstelle mit

- gemeinsamer Eigenschaft **size**
- abstrakten Methoden
- konkreten Methoden (unter Verwendung von abstrakten Methoden)

```
public abstract class List {
  private int size = 0;

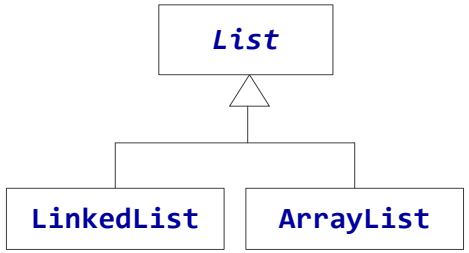
  protected List() {}

  public int size() { return size; }
  abstract Object get(int idx);
  public abstract int indexOf(Object o);
  public boolean contains(Object o) { return indexOf(o) >= 0; }
  public void clear() {…}
  public abstract void add(Object elem);
  public abstract void add(int idx, Object elem);
  public void addAll(Object... elems) {…}
  public abstract Object remove(int idx);
  public boolean remove(Object elem) {…}
  @Override
  public String toString() {…}

  // protected ----------------------------------------
  protected final void incrSize() { size++; }
  protected final void decrSize() { size--; }
  protected final void resetSize() { size = 0; }
  protected final void assertValidIndex(int idx) {…}
}
```

© H. Prähofer, 2026

Konkrete Klasse: LinkedList



- als doppelt-verkettete Liste
- mit Knotenklasse **Node**

```
public final class LinkedList extends AbstractList {
  private Node head;
  private Node tail;

  public LinkedList(Object... initial) {
    super();
    addAll(initial);
  }
  @Override
  public Object get(int idx) {
    assertValidIndex(idx);
    Node n = head;
    for (int i = 0; i < idx; i++) {
      n = n.next;
    }
    return n.elem;
  }
  ...
```

- Anmerkungen
  - ☐ Klasse ist **final**
  - ☐ Knotenklasse package-private und **final**
  - ☐ Verkettung schwierig
    ➔ Hilfsmethoden **link** und **unlink**

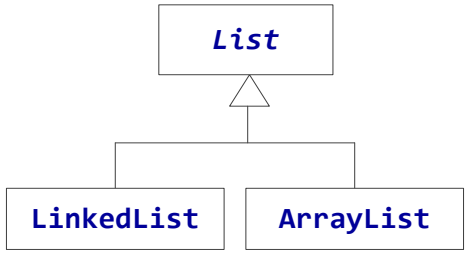
```
final class Node {
  Object elem;
  Node next, prev;

  Node(Object elem) {
    this.elem = elem;
  }
}
```

```
  @Override
  public void add(int idx, Object elem) {
    Node node = new Node(elem);
    if (idx == size()) {  // at end
      link(node, tail, null);
    } else {
      assertValidAddIndex(idx);
      Node p = head;
      for (int i = 0; i < idx; i++) { p = p.next; }
      link(node, p.prev, p);
    }
    incrSize();
  }
  ...
  private void link(Node node, Node prev, Node next) { … }
  private void unlink(Node node) {…}
}
```

© H. Prähofer, 2026

Konkrete Klasse: ArrayList



- Array zum Speichern der Elemente
- Füllung des Arrays gleich **size**

```
public final class ArrayList extends AbstractList {

  private static final int CAP = 16;
  private Object[] elems;

  public ArrayList(Object... initial) {
    super();
    elems = new Object[CAP];
    addAll(initial);
  }

  @Override
  public Object get(int idx) {
    assertValidIndex(idx);
    return elems[idx];
  }
  …
```

- Anmerkungen
  - Klasse is **final**
  - Vergrößern des Arrays, wenn voll

```
  @Override
  public void add(int idx, Object elem) {
    assertValidIndex(idx);
    if (elems.length == size()) {
      Object[] resized = new Object[size() * 2];
      System.arraycopy(elems, 0, resized, 0, size());
      elems = resized;
    }
    System.arraycopy(elems, idx, elems, idx + 1, size() - idx);
    elems[idx] = elem;
    incrSize();
  }

  @Override
  public void clear() {
    elems = new Object[CAP];
    resetSize();
  }
  ...
}
```

© H. Prähofer, 2026

Problemanalyse

Implementierung hat noch eine Reihe von Problemen

- Man kann beliebige Objekte speichern, beim Lesen sind aber **Typecasts** notwendig
  - Typcasts sind eine häufige Fehlerquelle

```
public class LinkedList extends AbstractList {
  protected Node head;
  ...
  public boolean add(Object elem) {...}
  public Object get(int idx) {...}
  ...
}
```

Parametertyp **Object** erlaubt beliebige Objekte

Rückgabetyp garantiert nur **Object**

```
final class Node {
  public Object elem;
  ...
}
```

Knotenwert von Typ **Object**

```
List list = new LinkedList();
```

add von beliebigen Objekten

```
list.add(new Circle(10, 10, 20));
list.add(new Rectangle(10, 10, 20, 30));
list.add(new Timer());
list.add("Hallo");
```

get mit Typcasts

```
Circle sp = (Circle) list.get(0);
Rectangle r = (Rectangle) list.get(1);
Timer t = (Timer) list.get(2);
String s = (String) list.get(3);
```

Fehlerquelle Typecasts: Wenn falscher Zugriff, dann ClassCastException
➔ **Abhilfe durch Generics**

© H. Prähofer, 2026

Problemanalyse

Implementierung hat noch eine Reihe von Problemen

- Man kann beliebige Objekte speichern, beim sind Lesen aber **Typecasts** notwendig
  - Typcasts sind eine häufige Fehlerquelle

➔ **Lösung: Generics (siehe Kapitel 6)**

- Spezialisierungen sind oft schwierig
  - Detailwissen der Basisklasse ist notwendig
  - Erweiterungen sind stark von Basisklasse abhängig
  - Basisklasse muss für Vererbung korrekt gestaltet sein
    ➔ Eingreifen in Verhalten der Basisklasse nicht möglich

➔ **Lösung**:
- Klassen mit einer **vollständigen Schnittstelle** schaffen, sodass weitere Methoden überflüssig
  ➔ konkrete Klassen dann **final** machen
- Zusätze bei Methoden dann **nicht durch Vererbung**, sondern durch **Delegation** realisieren
  ➔ **siehe Entwurfsmuster**

© H. Prähofer, 2026

Problemanalyse: Implementierung von Zusätzen

Beispiel

- Listen mit Logging der Methodenaufrufe

Probleme bei Zusätzen durch Vererbung

- Zusätze müssten sowohl für **LinkedList** als auch für **ArrayList** implementiert werden
- Vererbung ist oft schwierig und es kann zu unerwarteten Interaktionen zwischen Superklasse und Subklasse kommen

  ➔ siehe nächste Folie

© H. Prähofer, 2026

Problemanalyse: Implementierung von Zusätzen

Beispiel LoggingLinkedList

- Ableiten von **LinkedList**
- Methodenaufrufe mit Log-Ausgaben und super-Aufrufe zu Methode von LinkedList

```
public class LoggingLinkedList extends LinkedList {
    public LoggingLinkedList(Object... initial) {
        super(initial);
        log("Constructor called with " + Arrays.toString(initial));
    }
    ...
    @Override
    public void add(Object elem) {
        log("add called with " + elem);
        super.add(elem);
    }
    @Override
    public void addAll(Object... elems) {
        log("addAll called with " + Arrays.toString(elems));
        super.addAll(elems);
    }
    @Override
    public boolean remove(int idx) {
        log("remove called index " + idx);
        return super.remove(idx);
    }
    ...
    private void log(String msg) {...}
}
```

```
LinkedList l = new LoggingLinkedList("A", "B");
l.addAll("C", "D");
```

Gewünschter und erwarteter Log-Output:

```
Constructor called with [A, B]
addAll called with [C, D]
```

Tatsächlicher Log-Output:

```
addAll called with [A, B]
add called with A
add called with B
Constructor called with [A, B]
addAll called with [C, D]
add called with C
add called with D
```

!?

Ursache:

- Konstruktor ruft **addAll** auf
- **addAll** ruft **add** auf
- **add** und **addAll** sind dynamisch gebunden

© H. Prähofer, 2026

2. Vererbung und Dynamische Bindung

2.1 Vererbung bei Klassen

2.2 Typhierarchien und dynamische Bindung

2.3 Kovarianz bei Rückgabewerten und Exceptions

2.4 Fallstudie List

2.5 Entwurfsmuster

© H. Prähofer, 2026

Entwurfsmuster (Design Patterns)

Entwurfsmuster (Design Patterns) sind schematische Lösungen für häufig wiederkehrende Probleme

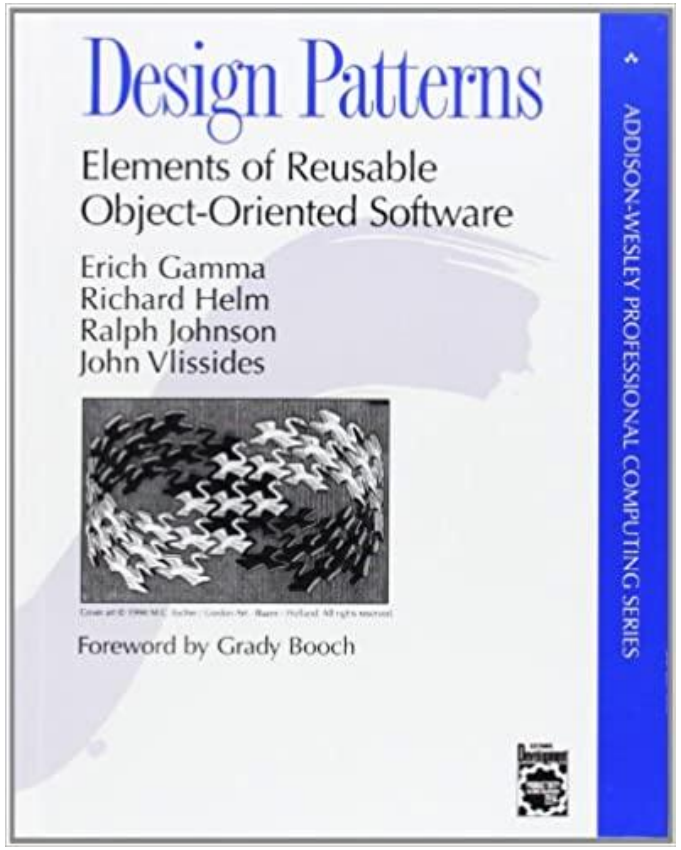
Objektorientierte Entwurfmuster:

- Objektstrukturen, d.h. Zusammenwirken von mehreren Objekten,
- um bestimmte Aufgabenstellungen objektorientiert zu lösen
- arbeiten mit
  - Vererbung
  - und dynamischer Bindung

© H. Prähofer, 2026

Design Patterns: Elements of Reusable Object-Oriented Software

- Buch, das den Begriff "Design Pattern" prägte
- entstand im Kontext von Frameworks für graphische Oberflächen



erschienen 1995
ein Bestseller der Informatik-Literatur

enthält 23 Muster

klassifiziert in

- Erzeugende Muster
- Strukturmuster
- Verhaltensmuster

© H. Prähofer, 2026

Bestandteile eines Musters

1. **Name**: Schafft Begriffe und definiert ein Vokabular

   z.B. "Dekorator", "Adapter", "Fabrik", ...

Vokabular des Softwareentwicklers

2. **Zweck**: Wann soll man das Muster verwenden?

   z.B. Flexibles Hinzufügen von Zusätzen ohne Klassen ableiten zu müssen

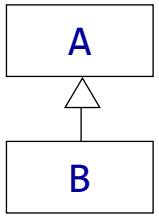
3. **Lösung**: Objektstrukturen und Interaktionen

   z.B. Klassenhierarchie mit Decorator-Klassen (siehe folgend)

© H. Prähofer, 2026

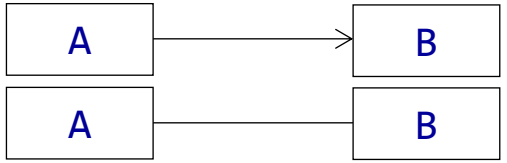
Exkurs: UML Notation

- **Generalisierung/Spezialisierung**



B ist ein *A*
B erweitert *A*

- **Assoziation**: steht-in-Bezug Beziehung



A benutzt *B*

A und *B* stehen in Beziehung

- **Aggregation**: hat-Beziehung



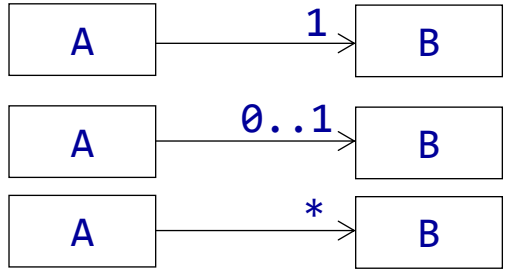
A hat ein *B*

- **Komposition:** besteht-aus-Beziehung



A setzt sich zusammen aus *B*

- **Kardinalität**



genau ein *B*

kein oder ein *B*

beliebig viele *Bs*

verwendbar bei
- Assoziation,
- Aggregation
- Komposition

© H. Prähofer, 2026

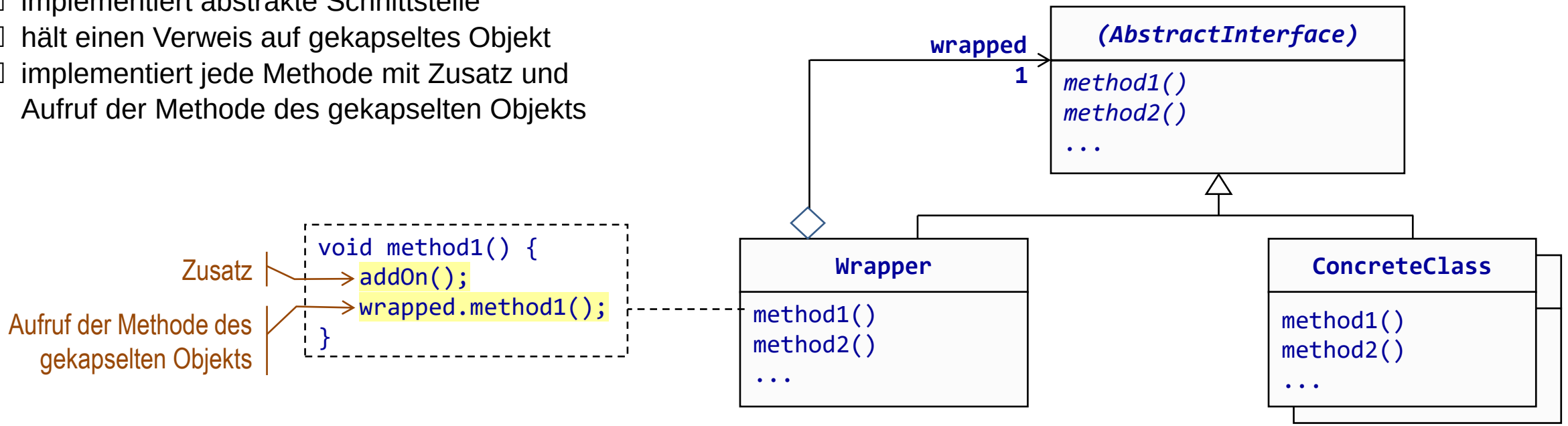
Entwurfsmuster Wrapper

Zweck

- Objekte kapseln und damit mit einem Zusatz versehen

Lösung

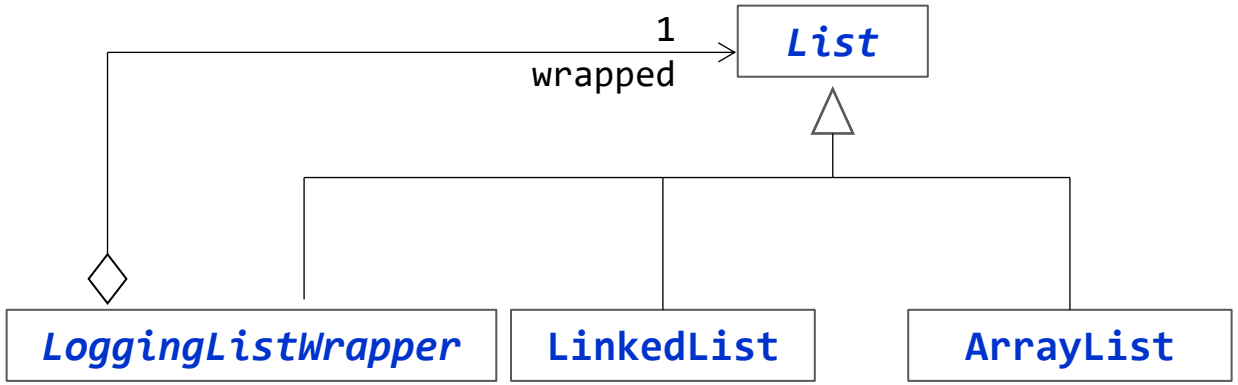
- Definiere für die Klasse eine abstrakte Schnittstelle
- Implementiere die Schnittstelle durch die Klasse
- Implementiere Wrapper-Klasse folgend:
  - ☐ implementiert abstrakte Schnittstelle
  - ☐ hält einen Verweis auf gekapseltes Objekt
  - ☐ implementiert jede Methode mit Zusatz und Aufruf der Methode des gekapselten Objekts



© H. Prähofer, 2026

Entwurfsmuster Wrapper: LoggingListWrapper

Beispiel: LoggingListWrapper



© H. Prähofer, 2026

Entwurfsmuster Wrapper: LoggingListWrapper

Beispiel: LoggingListWrapper

```
public class LoggingListWrapper extends List {

  private final List wrapped;

  public LoggingListWrapper(List wrapped) {
    log("Constructor called ");
    this.wrapped = wrapped;
  }
  ...
  @Override
  public void add(Object elem) {
    log("add called with element " + elem );
    wrapped.add(elem);
  }
  @Override
  public void addAll(Object... elems) {
    log("addAll called with elements %s".formatted(elems));
    wrapped.addAll(elems);
  }

  ...
}
```

gekapseltes Objekt

Zusatz log

```
List l = new LoggingListWrapper(
                        new LinkedList());

l.addAll("A", "B", "C");
l.add("D");
l.remove("C");
```

```
Constructor called
addAll called with [A, B, C]
add called with D
remove called with C
```

```
List l = new LoggingListWrapper(
                        new ArrayList());
...
```

© H. Prähofer, 2026

2. Vererbung

2.1 Vererbung bei Klassen

2.2 Typhierarchien und dynamische Bindung

2.3 Ausnahmebehandlung und Exception-Klassen

2.4 Kovarianz bei Rückgabewerten und Exceptions

2.5 Fallstudie List

2.6 Entwurfsmuster

- ☐ Wrapper
- ☐ Decorator
- ☐ Adapter
- ☐ Factory

2.7 Zusammenfassung

© H. Prähofer, 2026

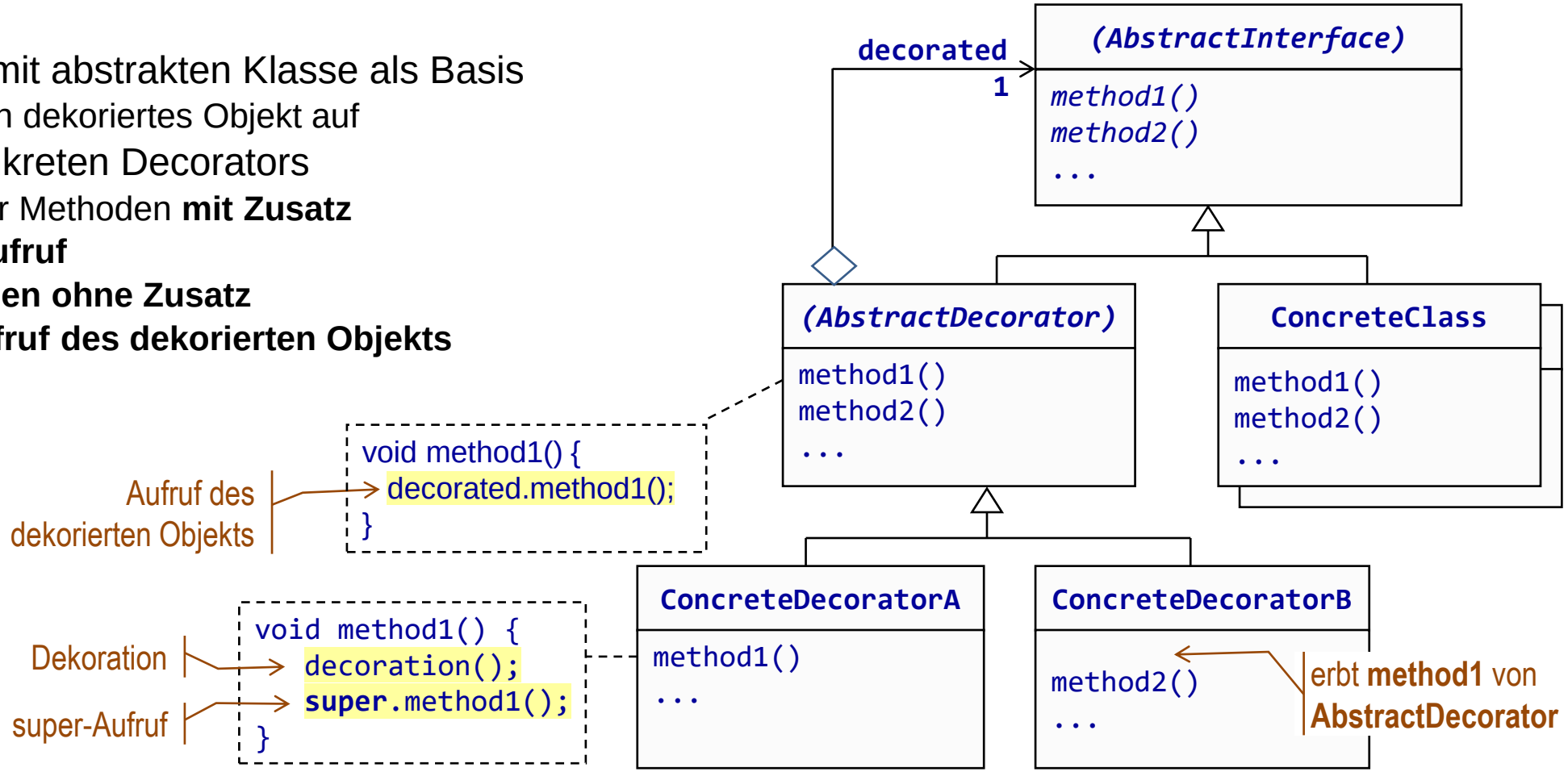
Entwurfsmuster Decorator

Zweck

- Neues Verhalten zu einer Klasse hinzufügen, ohne die Klasse zu ändern und ohne eine Unterklasse zu bilden

Lösung

- wie Wrapper, aber mit abstrakten Klasse als Basis
  - alle Methoden rufen dekoriertes Objekt auf
- Ableitung eines konkreten Decorators
  - **Überschreiben** der Methoden **mit Zusatz**
    - **mit Super-Aufruf**
  - **Erben der Methoden ohne Zusatz**
    - damit nur **Aufruf des dekorierten Objekts**



© H. Prähofer, 2026

Entwurfsmuster Decorator: ListDecorator

Beispiel ListDecorator

```
public abstract class AbstractListDecorator extends List {

  private final List decorated;

  public ListDecorator(List decorated) {
    this.decorated = decorated;
  }

  @Override
  public int size() {
    return decorated.size();
  }

  @Override
  public Object get(int idx) {
    return decorated.get(idx);
  }

  @Override
  public void add(Object elem) {
    decorated.add(elem);
  }
  ...
}
```

Kette von Decorators mit LinkedList am Ende!

```
public class LoggingListDecorator extends AbstractListDecorator {
  public LoggingListDecorator(List decorated) {
    super(decorated);
  }
  @Override
  public int size() {
    log("size called ");
    return super.size();
  }
  ...
}
```

mit Super-Aufruf wird das dekorierte Objekt aufgerufen!

```
public class NullCheckDecorator extends AbstractistDecorator {
  public NullCheckDecorator(List decorated) {
    super(decorated);
  }
  @Override
  public int indexOf(Object o) {
    if (o == null) return -1;
    return super.indexOf(o);
  }
  @Override
  public void add(Object elem) {
    if (elem == null) return;
    super.add(elem);
  }
}
```

Alle anderen Methoden von **ListDecorator** geerbt

```
List decoratedList =
    new NullCheckDecorator(
            new LoggingListDecorator(
                    new LinkedList()));
```

© H. Prähofer, 2026

Entwurfsmuster Decorator: UnmodifiableList

Beispiel UnmodifiableList

- Decorator, um Veränderungen zu verhindern

```
public class UnmodifiableListDecorator extends AbstractistDecorator {

  public UnmodifiableListDecorator(List decorated) {
    super(decorated);
  }

  @Override
  public void add(Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public void add(int idx, Object elem) {
    throw new UnsupportedOperationException("forbidden");
  }

  @Override
  public Object remove(int idx) {
    throw new UnsupportedOperationException("forbidden");
  }

  ...
}
```

Anwendung: **nur lesender** Zugriff auf interne Daten

```
public class MyClass {

    private List myPrivateListData = new ArrayList();

    public List getListData() {
        return new UnmodifiableListDecorator(myPrivateListData);
    }

    ...
}
```

Nur lesende Zugriffe !

```
MyClass myObject = new MyClass();
List data = myObject.getListData();

data.remove(...); // throws UnsupportedOperationException
```

Problematisch: **List** bietet **add**, **remove** etc. an, und **UnmodifiableList** verhindert diese !?

© H. Prähofer, 2026

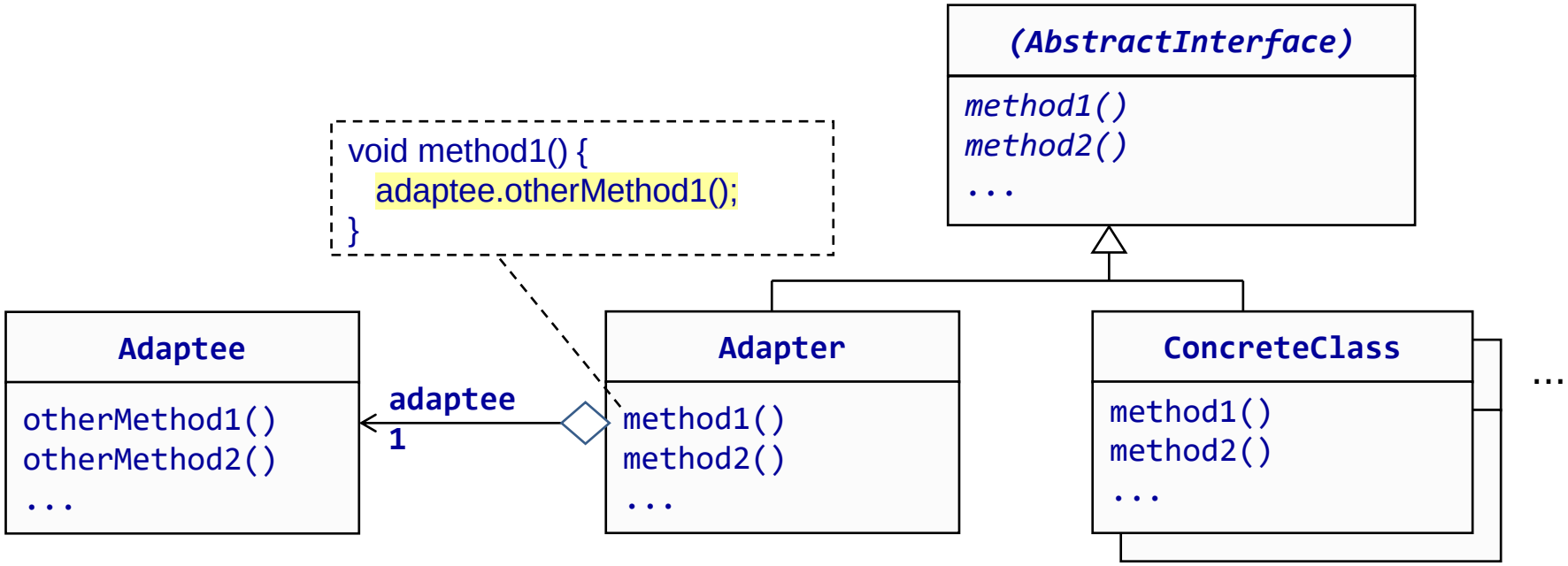
Entwurfsmuster Adapter

Zweck

- Fremde Klasse zu einem bestehenden Klassensystem kompatibel machen

Lösung

- Gemeinsames Interface
- Klasse, die Interface implementiert und Aufrufe in Aufrufe an die fremde Klasse umsetzt (Forwarding)

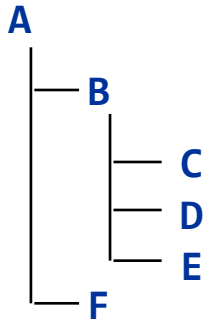


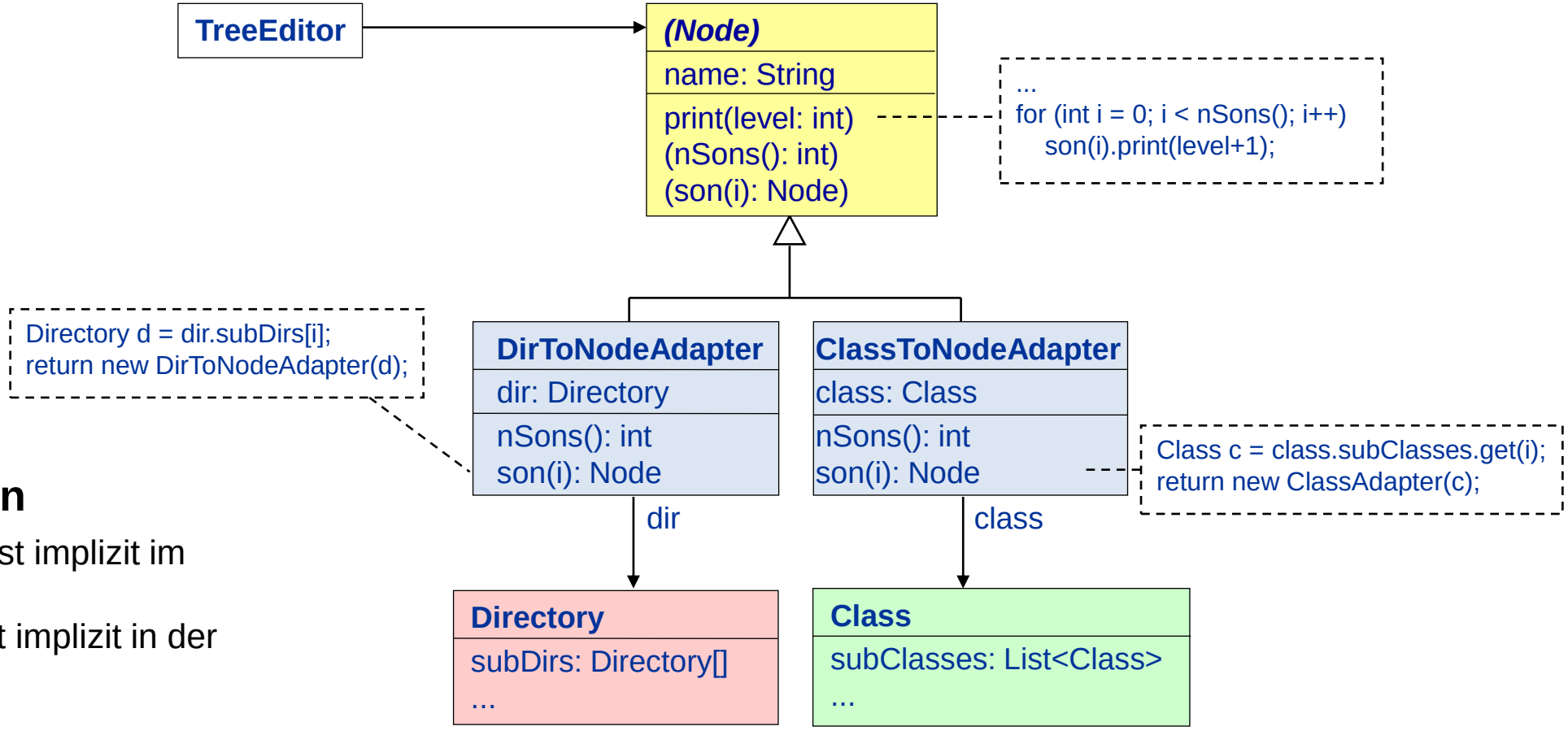
© H. Prähofer, 2026

Entwurfsmuster Adapter

Beispiel Adapter: Klassendiagramm für Baumknoten-Adapter

- Einheitliche Schnittstelle für verschiedene Baumstrukturen





Unterschiedliche Daten

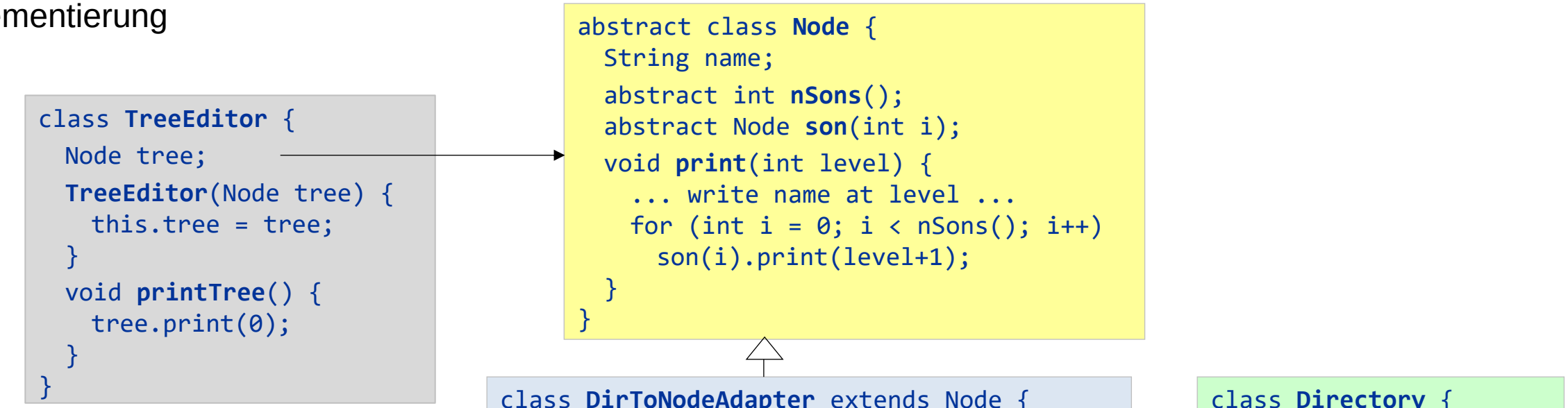
- *Verzeichnisstruktur*: Baum ist implizit im Dateisystem abgebildet
- *Klassenhierarchie*: Baum ist implizit in der Typinformation gespeichert
- ...

© H. Prähofer, 2026

Entwurfsmuster: Adapter [3/3]

Beispiel Adapter: Klassendiagramm für Baumknoten-Adapter

- Implementierung



- Benutzung

```
TreeEditor editor =
  new TreeEditor(
    new DirToNodeAdapter(
      Directory.rootDirectory()
    )
  );
...
editor.printTree();
```

© H. Prähofer, 2026

Entwurfsmuster Factory-Methode [1/2]

Zweck

- Abstrahiere von der Erzeugung von konkreten Objekten
- Verberge, wie Objekte erzeugt werden

Lösung

- Verberge den Konstruktor der konkreten Objekte
- Stelle für die Erzeugung von Objekten Methoden bereit
- Optional: Führe abstrakte Schnittstellen / abstrakte Klassen ein, um die konkreten Implementierungen zu verbergen

Konsequenzen

- Implementierungen können ausgetauscht werden
- erzeugte Strukturen können variiert und optimiert werden
- Factory-Methoden können flexibel auf Anforderungen reagieren

© H. Prähofer, 2026

Entwurfsmuster Factory-Methode [1/2]

Implementierung in Java

- Factory-Methode als statische Methoden
- oft mit Namen **of**

Beispiel: List.of

- überladene Methoden **List.of** erzeugen optimierte Listenstrukturen mit fixer Länge

```
List<String> none = List.of();
List<String> one = List.of("A");
List<String> two = List.of("A", "B");
List<String> many = List.of("A", "B", "C", "D", "E");
```

Singleton-Objekt

Objekt mit einem Feld

Objekt mit zwei Feldern

Objekt mit Array fixer Länge

hochoptimiert!

© H. Prähofer, 2026

Factory-Methode

Beispiel Shapes

- Factory-Methoden für konkrete Typen von **Shape**
- mit Klassenmethoden entweder in Basisklasse **Shape**

```
public abstract class Shape {

  public static Circle circle(int x, int y, int r) {
    return new Circle(x, y, r);
  }
  public static Rectangle rect(int x, int y, int w, int h) {
    return new Rectangle(x, y, w, h);
  }
  …
}
```

- oder in eigenen Utility-Klasse **Shapes**

Mehrzahl !

```
public class Shapes {

  public static Circle circle(int x, int y, int r) {…}

  public static Rectangle rect(int x, int y, int w, int h) {…}
}
```

```
public class Circle extends Shape {
  private int radius;

  public Circle(int x, int y, int r) {
    super(x, y);
    this.radius = r;
  }
  …
}
```

```
public class Rectangle extends Shape {
  private int w, h;

  public Rectangle(int x, int y, int w, int h) {
    super(x, y);
    this.w = w; this.h = h;
  }
  …
}
```

© H. Prähofer, 2026

Entwurfsmuster Adapter: ArrayToListAdapter

Beispiel: ArrayToListAdapter

- Array als **List** verwenden

```
final class ArrayToListAdapter extends List {
    private final Object[] array;

    public ArrayToListAdapter(Object[] array) {
        this.array = array;
    }
    @Override
    public int size() {
        return array.length;
    }
    @Override
    public Object get(int idx) {
        if (idx < 0 || idx >= array.length)
            throw new IndexOutOfBoundsException(idx);
        return array[idx];
    }
    @Override
    public int indexOf(Object o) {
        for (int i = 0; i < array.length; i++) {
            if (array[i].equals(o)) return i;
        }
        return -1;
    }
    ...
}
```

- Erzeugen mit Factory-Methode **of**

```
public class List {

    public static List of(Object[] array) {
      Objects.requireNonNull(array, "no null");
      return new ArrayToListAdapter(array);
    }

    public int size();
    ...
}
```

Factory-Methode, die Adapter für array erzeugt

```
Object[] array = {"A", "B", "C"};
List listFromArray = List.of(array);

System.out.println(listFromArray.get(0));
```

© H. Prähofer, 2026

Entwurfsmuster Adapter: Single2ListAdapter

Beispiel Adapter: Single2ListAdapter

- einzelnes Objekt als **List** verwenden

```
final class SingleToListAdapter extends List {
    private final Object single;

    public SingleToListAdapter(Object object) {
        this.single = object;
    }

    @Override
    public int size() {
        return 1;
    }

    @Override
    public Object get(int idx) {
        if (idx == 0) return single;
        else throw new IndexOutOfBoundsException(idx);
    }

    @Override
    public int indexOf(Object o) {
        if (o.equals(single)) return 0;
        else return -1;
    }
    ...
}
```

- Erzeugen mit Factory-Methode **of**

```
public class List {

    public static List of(Object[] array) {
        Objects.requireNonNull(array, "no null");
        return new Array2ListAdapter(array);
    }

    public static List of(Object single) {
      Objects.requireNonNull(single, "no null");
      return new Single2ListAdapter(single);
    }

    public int size();
    public Object get(int idx);
    public default boolean contains(Object o) {
        return indexOf(o) >= 0;
    }
    public int indexOf(Object o);
}
```

Factory-Methode, die Adapter für Objekt erzeugt

```
List listWithX = List.of("X");
System.out.println(listFromArray.get(0));
```

© H. Prähofer, 2026

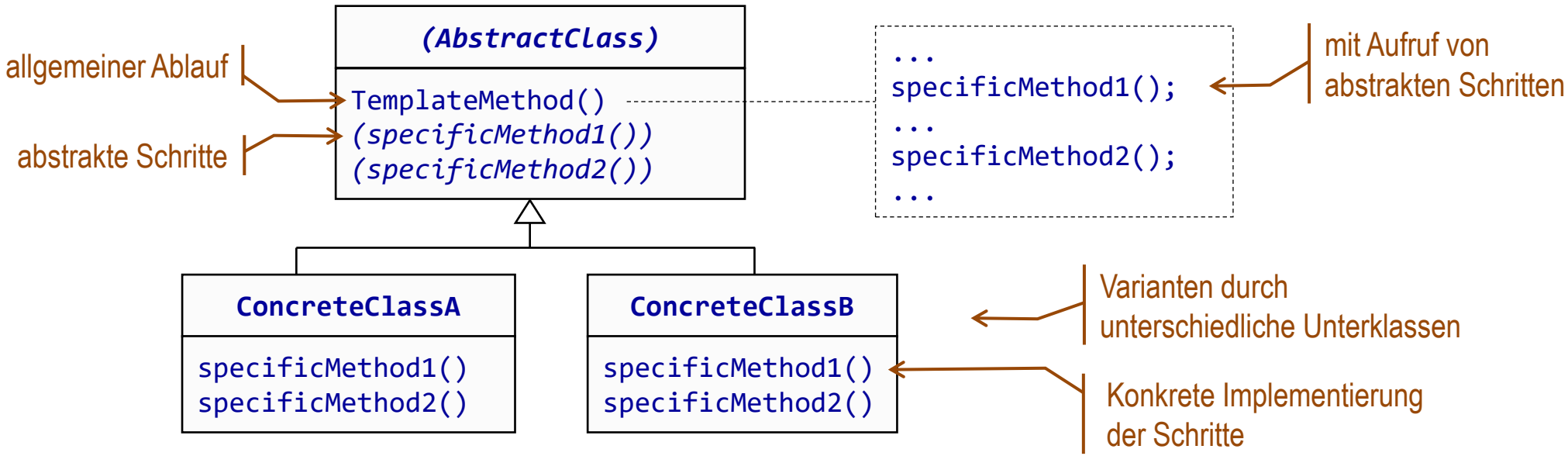
Template Method (Schablonenmethode)

Zweck

- Gerüst eines Algorithmus, bei dem die Schritte unterschiedlich ausgeprägt sein können

Lösung

- Definiere eine Methode für den allgemeinen Ablauf (Template) in abstrakten Basisklasse
- mit Schritten, die in Unterklassen speziell implementiert werden sollen
- Unterschiedliche Unterklassen erlauben allgemeinen Ablauf unterschiedlich auszuprägen



© H. Prähofer, 2026

Template Method (Schablonenmethode)

Beispiel Template Methode: DataProcessor

- konkreter Ablauf **process** mit Schritten Laden, Verarbeiten und Schreiben der Daten
- abstrakte Methode **parseContent** zum Verarbeiten der Daten

```
public abstract class DataProcessor {

  public final void process(String source) {
    String content = loadData(source);
    Info info = parseContent(content);
    writeInfo(info);
  }

  protected String loadData(String source) { ... }

  protected abstract Info parseContent(String data);

  protected void writeInfo(Info info) {
    info.write();
  }
}
```

abstrakten Basisklasse mit

konkreter Methode für allgemeinen Ablauf

konkreter Methode zum Laden der Daten

abstrakter Methode zur Verarbeitung

konkreter Methode zum Schreiben der Ergebnisse

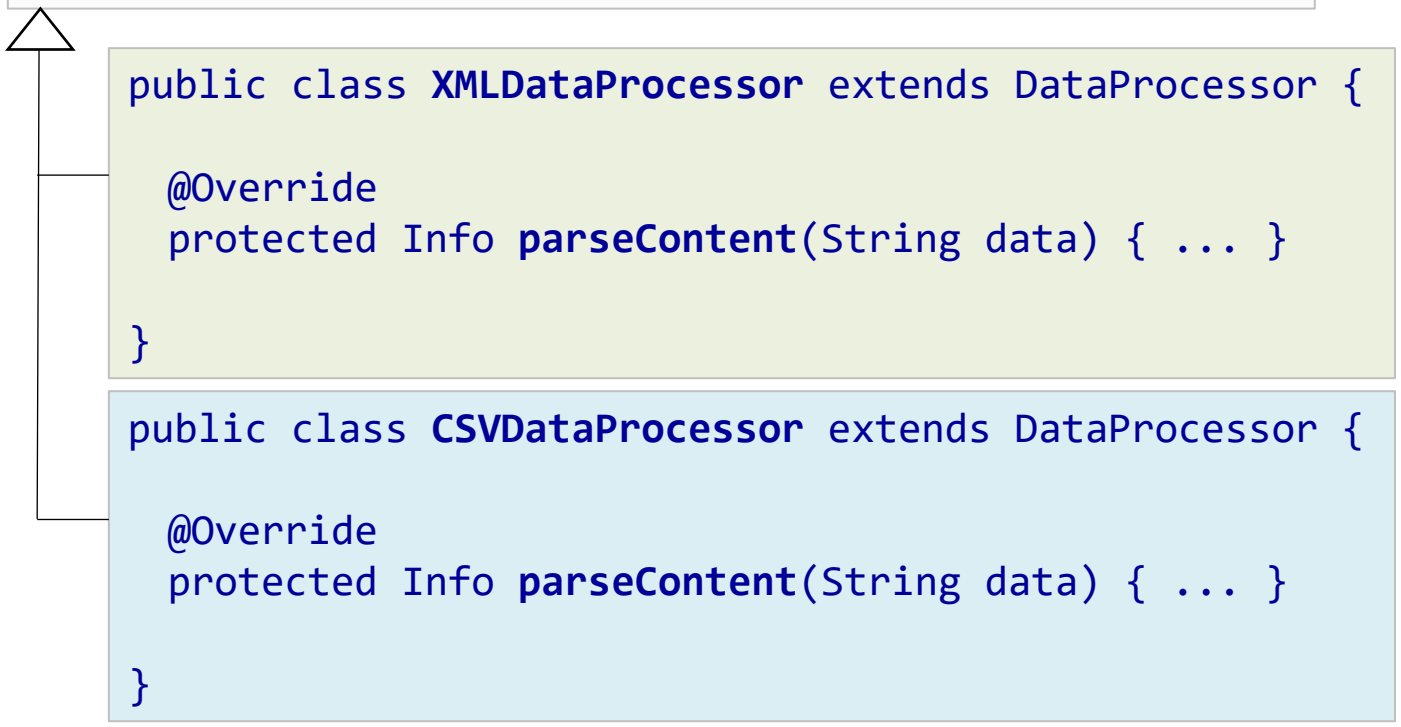
© H. Prähofer, 2026

Template Method (Schablonenmethode)

Beispiel Template Method: DataProcessor

- Ausprägung der Methode **parseContent** in Unterklassen

```
public abstract class DataProcessor { ... }
```



```
public class XMLDataProcessor extends DataProcessor {

  @Override
  protected Info parseContent(String data) { ... }

}
```

Konkrete Ausprägung zur Verarbeitung von XML Daten

mit konkreter Methode zur Verarbeitung der XML Daten

```
public class CSVDataProcessor extends DataProcessor {

  @Override
  protected Info parseContent(String data) { ... }

}
```

Konkrete Ausprägung zur Verarbeitung von CSV Daten

mit konkreter Methode zur Verarbeitung der CSV Daten

© H. Prähofer, 2026

Template Methode CopyTo bei List

- Methode **copyTo** verwendet **size**, **add** und **get** unabhängig von konkreten Implementierung der beiden Listen **from** und **to**

```
public abstract class List {
  …
  public static void copyTo(List from, List to) {
    for (int i = 0; i < from.size(); i++) {
      to.add(from.get(i));
    }
  }
  …
}
```

© H. Prähofer, 2026

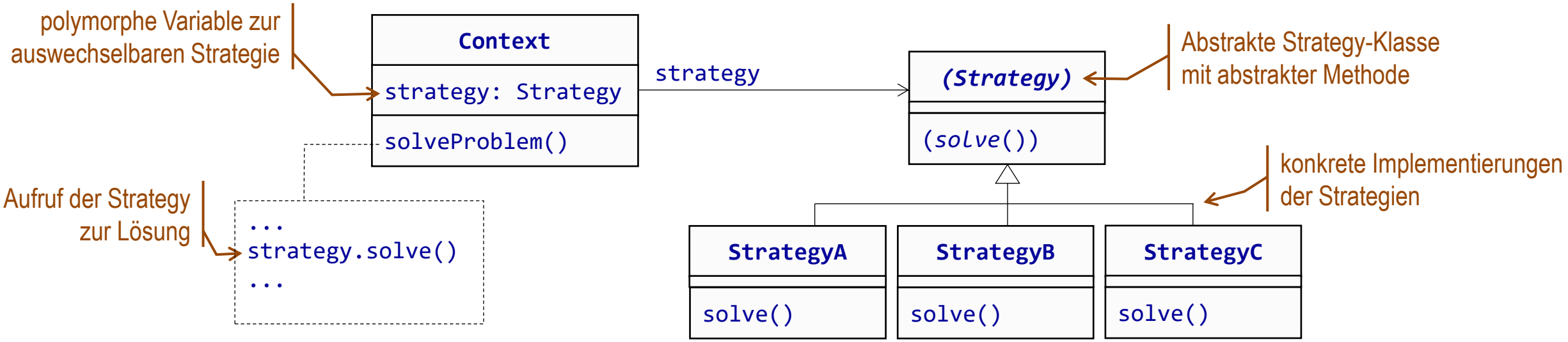
Strategy (Strategie)

Zweck

- Definiere eine Familie von Verfahren, die man beliebig austauschen kann

Lösung

- Lagere das Verfahren in ein eigenes Strategie-Objekt aus
- Definiere eine abstrakte Klasse mit der Schnittstelle für das Verfahren
- Implementiere unterschiedliche Ausprägungen des Verfahrens in Unterklassen
- Verwende eine Variable im Kontext, mit der man die Variante des Verfahren definieren kann



© H. Prähofer, 2026

Strategy (Strategie)

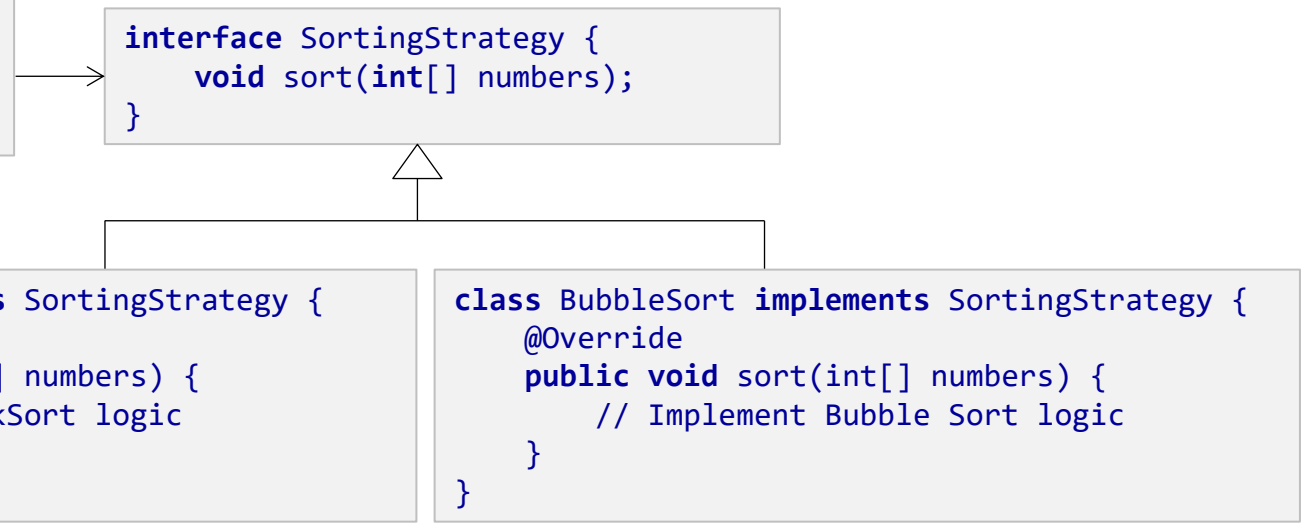
Beispiel: Sortieralgorithmus

```
class Application {
 public static void main(String[] args) {
    int[] data = {5, 3, 8, 1, 2, ...};
    setStrategy(new BubbleSort());
    sort(data);
  }

  public static void setStrategy(SortingStrategy strategy) {
    this.strategy = strategy;
  }

  public static void sort(int[] numbers) {
    strategy.sort(numbers);
  }

  private SortingStrategy strategy;
}
```



© H. Prähofer, 2026

Strategy (Strategie)

Beispiel: Loader

- Laden der Daten durch Strategie-Objekt **Loader**
- mit Unterklassen **FileLoader** und **NetLoader**

```
public abstract class DataProcessor {
  private final Loader loader;

  public DataProcessor(Loader loader) {
    this.loader = loader;
  }

  protected String loadData(String source) {
    return loader.load(source);
  }
  ...
}
```

```
...
DataProcessor p =
         new XMLDataProcessor(new NetLoader());
p.process("https://.../data.xml");
```

Setzen der Strategie

```
public abstract class Loader {
  public abstract String load(String source);
}
```

```
public class FileLoader extends Loader {
  @Override
  public String load(String src) {
    try {
      return Files.readString(Paths.get(src));
    } catch (IOException e) {
      return "";
    }
  }
}
```

```
public class NetLoader extends Loader {
  @Override
  public String load(String src) { ... }
}
```

List: Methode Sort mit Strategie für Sortierung

- Sortierung mit Sortierstrategie **sortingStrategy** als Parameter

```
public interface Comparator<T> {
   int compare(T o1, T o2);
   …
}
```

```
public abstract class List {
  …
  public <T> void sort(Comparator<Object> sortingStrategy) {
    for (int i = 1; i < size(); i++) {
      Object element = get(i);
      int insertionIndex = i;
      while (insertionIndex > 0 &&
                  sortingStrategy.compare(get(insertionIndex - 1), element) > 0) {
        insertionIndex--;
      }
      if (insertionIndex != i) {
        remove(i);
        add(insertionIndex, element);
      }
    }
  }
  …
}
```

Strategy (Strategie)

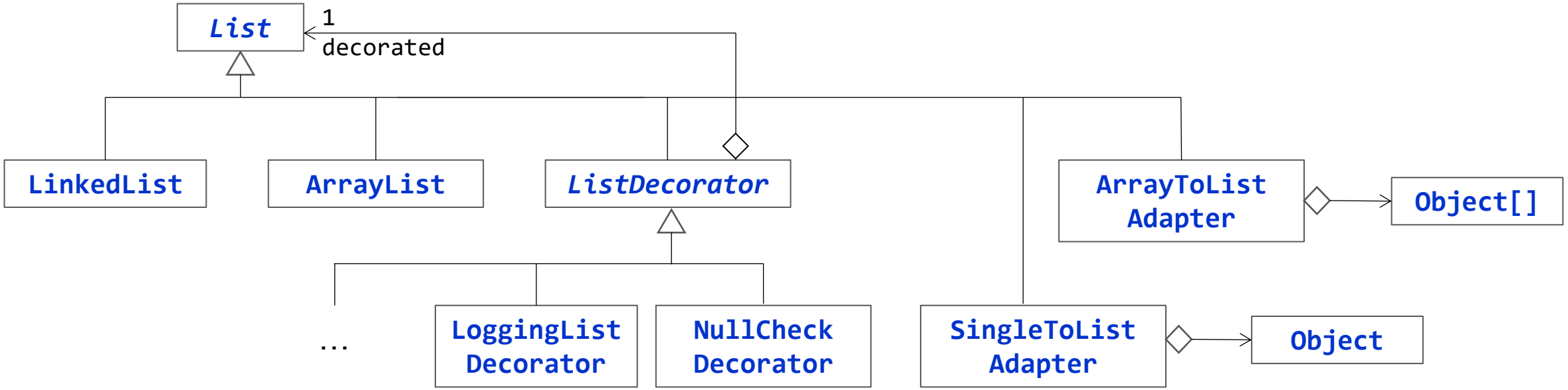
Anwendungsbeispiele

- Algorithmen
  - Sortieralgorithmen (z. B. QuickSort, MergeSort, BubbleSort)
  - Suchalgorithmen (z. B. Binäre Suche, Lineare Suche)
  - Pfadfindungsstrategien (z. B. A*, Dijkstra's Algorithmus)
  - …
- Serialisierungs-Strategien: JSON, XML, YAML, ...
- Datenkomprimierung: ZIP, RAR, GZIP, 7z
- Logging-Strategien: Protokollierung in Konsole, Datei, Datenbank, Cloud-Dienste
- Authentifizierungsmechanismen: OAuth, JWT, LDAP, SAML
- Textverarbeitung
  - Rechtschreibprüfung (z. B. Hunspell, LanguageTool)
  - Textkodierungsformate (z. B. Base64, UTF-8, ASCII)
- uvm.

© H. Prähofer, 2026

Zusammenfassung Fallstudie List

Klassendiagramm der aktuellen Implementierung



Vorschau auf weitere Verbesserungen

- Interfaces mit Unterscheidung read-only und mutable
- Innere Klassen für Knoten und Iteratoren
- Generischer Typ für Werte

© H. Prähofer, 2026

Lernziele überprüfen

- Was sind Entwurfsmuster?
- Erläutern Sie den Zweck und die Lösungsstruktur für folgende Entwurfsmuster
  - ☐ Wrapper
  - ☐ Decorator
  - ☐ Adapter
  - ☐ Factory Method
  - ☐ Template Method
  - ☐ Strategy

© H. Prähofer, 2026